

PROJECT DOCUMENTATION

# LLM120

## Complete Project Manual

From first setup to model architecture, data preparation, pretraining, assistant tuning, and maintenance.

---

### Code-reviewed edition

Repository: 120r-copied

Package: llm120 0.1.0

Review date: 16 September 2026

### Evidence boundary

Based on every source file, configuration, test, and notebook cell in the supplied snapshot. Includes verified code behavior, operating recipes, exact references, and unresolved findings. No trained model or end-to-end GPU result was available for this review.

# Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1. Scope, evidence, and reading guide</b>             | <b>1</b>  |
| <b>2. What the project builds</b>                        | <b>1</b>  |
| 2.1 Concepts from zero . . . . .                         | 1         |
| 2.2 End-to-end artifact flow . . . . .                   | 2         |
| 2.3 What success means . . . . .                         | 2         |
| <b>3. Repository layout and software environment</b>     | <b>2</b>  |
| 3.1 Dependencies and installation semantics . . . . .    | 3         |
| 3.2 Local setup . . . . .                                | 3         |
| 3.3 Path rules . . . . .                                 | 3         |
| <b>4. Architecture and resource planning</b>             | <b>3</b>  |
| 4.1 Decoder architecture . . . . .                       | 3         |
| 4.2 Token and time arithmetic . . . . .                  | 4         |
| 4.3 Memory and checkpoint storage . . . . .              | 4         |
| <b>5. First end-to-end pilot</b>                         | <b>5</b>  |
| <b>6. Downloading and preparing the corpus</b>           | <b>5</b>  |
| 6.1 Download behavior . . . . .                          | 5         |
| 6.2 Tokenizer training . . . . .                         | 5         |
| 6.3 Binary preparation . . . . .                         | 6         |
| 6.4 Reading tokens during training . . . . .             | 6         |
| <b>7. Configuration reference and experiment design</b>  | <b>7</b>  |
| 7.1 Settings that control identity . . . . .             | 7         |
| 7.2 Training settings . . . . .                          | 7         |
| <b>8. Pretraining internals and execution</b>            | <b>7</b>  |
| 8.1 Starting a run . . . . .                             | 7         |
| 8.2 One optimizer step . . . . .                         | 8         |
| 8.3 Learning-rate schedule . . . . .                     | 8         |
| 8.4 Evaluation and logging . . . . .                     | 8         |
| <b>9. Checkpoints, stopping, and reproducibility</b>     | <b>8</b>  |
| 9.1 Base checkpoint structure . . . . .                  | 8         |
| 9.2 Graceful stopping and resuming . . . . .             | 9         |
| 9.3 What identity checks cover . . . . .                 | 9         |
| <b>10. Evaluating and generating from the base model</b> | <b>9</b>  |
| <b>11. Assistant data and normalization</b>              | <b>10</b> |
| 11.1 Sources and files . . . . .                         | 10        |
| 11.2 SFT normalization . . . . .                         | 10        |
| 11.3 DPO normalization . . . . .                         | 10        |
| 11.4 HelpSteer and RL prompts . . . . .                  | 10        |
| <b>12. Assistant training stages</b>                     | <b>11</b> |
| 12.1 SFT . . . . .                                       | 11        |
| 12.2 DPO . . . . .                                       | 11        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 12.3 Reward model . . . . .                                | 12        |
| 12.4 RLOO . . . . .                                        | 12        |
| 12.5 Outputs, smoke runs, and resume differences . . . . . | 12        |
| 12.6 Chat behavior . . . . .                               | 12        |
| <b>13. Google Colab path</b>                               | <b>13</b> |
| 13.1 Notebook sequence . . . . .                           | 13        |
| 13.2 Blocking issues before relying on Colab . . . . .     | 13        |
| <b>14. Tokenizer comparison and multilingual extension</b> | <b>13</b> |
| 14.1 Comparison script . . . . .                           | 13        |
| 14.2 Nepali adaptation . . . . .                           | 14        |
| <b>15. Troubleshooting and operational checklist</b>       | <b>14</b> |
| <b>16. Tests and validation performed for this manual</b>  | <b>15</b> |
| 16.1 Included tests . . . . .                              | 15        |
| 16.2 Review execution results . . . . .                    | 15        |
| <b>17. Review findings and maintenance priorities</b>      | <b>15</b> |
| 17.1 Address before a constrained or Colab run . . . . .   | 15        |
| 17.2 Reproducibility and integrity gaps . . . . .          | 16        |
| 17.3 Data and edge cases . . . . .                         | 16        |
| 17.4 Suggested development order . . . . .                 | 16        |
| <b>18. Developer walkthrough and extension boundaries</b>  | <b>17</b> |
| 18.1 Module responsibilities . . . . .                     | 17        |
| 18.2 Adding a model or dataset . . . . .                   | 17        |
| 18.3 Export and serving boundary . . . . .                 | 18        |
| 18.4 Rebuilding this documentation . . . . .               | 18        |
| <b>A. Complete command-line reference</b>                  | <b>19</b> |
| A. llm120-download . . . . .                               | 19        |
| A. llm120-tokenizer . . . . .                              | 19        |
| A. llm120-prepare . . . . .                                | 19        |
| A. llm120-train . . . . .                                  | 20        |
| A. llm120-generate . . . . .                               | 20        |
| A. llm120-benchmark . . . . .                              | 20        |
| A. llm120-doctor . . . . .                                 | 21        |
| A. llm120-download-posttrain . . . . .                     | 21        |
| A. llm120-prepare-posttrain . . . . .                      | 21        |
| A. llm120-sft . . . . .                                    | 21        |
| A. llm120-dpo . . . . .                                    | 21        |
| A. llm120-reward . . . . .                                 | 22        |
| A. llm120-rlhf . . . . .                                   | 22        |
| A. llm120-chat . . . . .                                   | 22        |
| A. Standalone tokenizer comparison . . . . .               | 23        |
| <b>B. Every shipped YAML configuration</b>                 | <b>23</b> |
| B. pilot.yaml . . . . .                                    | 23        |
| B. train_125m.yaml . . . . .                               | 24        |
| B. train_300m.yaml . . . . .                               | 24        |

|                                                   |           |
|---------------------------------------------------|-----------|
| B. train_350m.yaml . . . . .                      | 25        |
| B. train_370m.yaml . . . . .                      | 26        |
| B. train_400m.yaml . . . . .                      | 26        |
| <b>C. Post-training constructor settings</b>      | <b>27</b> |
| C. sft.py . . . . .                               | 27        |
| C. dpo.py . . . . .                               | 28        |
| C. reward.py . . . . .                            | 28        |
| C. rlhf.py . . . . .                              | 29        |
| <b>D. Complete Python symbol index</b>            | <b>30</b> |
| D. src/llm120/__init__.py . . . . .               | 30        |
| D. src/llm120/benchmark.py . . . . .              | 30        |
| D. src/llm120/chat.py . . . . .                   | 30        |
| D. src/llm120/checkpoint.py . . . . .             | 30        |
| D. src/llm120/config.py . . . . .                 | 30        |
| D. src/llm120/data.py . . . . .                   | 30        |
| D. src/llm120/doctor.py . . . . .                 | 31        |
| D. src/llm120/download.py . . . . .               | 31        |
| D. src/llm120/dpo.py . . . . .                    | 31        |
| D. src/llm120/generate copy.py . . . . .          | 31        |
| D. src/llm120/generate.py . . . . .               | 31        |
| D. src/llm120/model.py . . . . .                  | 31        |
| D. src/llm120/posttrain_common.py . . . . .       | 32        |
| D. src/llm120/posttrain_download.py . . . . .     | 32        |
| D. src/llm120/posttrain_prepare.py . . . . .      | 32        |
| D. src/llm120/prepare.py . . . . .                | 32        |
| D. src/llm120/reward.py . . . . .                 | 33        |
| D. src/llm120/rlhf.py . . . . .                   | 33        |
| D. src/llm120/schedule.py . . . . .               | 33        |
| D. src/llm120/sft.py . . . . .                    | 33        |
| D. src/llm120/tokenizer_train.py . . . . .        | 33        |
| D. src/llm120/train.py . . . . .                  | 33        |
| D. scripts/compare_tokenizers.py . . . . .        | 34        |
| D. tests/test_data.py . . . . .                   | 34        |
| D. tests/test_doctor.py . . . . .                 | 34        |
| D. tests/test_download.py . . . . .               | 34        |
| D. tests/test_posttrain.py . . . . .              | 34        |
| D. tests/test_schedule.py . . . . .               | 34        |
| <b>E. Colab cell-by-cell reference</b>            | <b>34</b> |
| E. Source-copy equivalence . . . . .              | 35        |
| <b>F. Packaging and recorded dependencies</b>     | <b>35</b> |
| F. pyproject.toml . . . . .                       | 35        |
| F. Colab requirements . . . . .                   | 36        |
| F. Lockfile package inventory . . . . .           | 36        |
| <b>G. Review coverage and source fingerprints</b> | <b>37</b> |
| <b>H. Glossary and handoff checklist</b>          | <b>39</b> |

# 1. Scope, evidence, and reading guide

This manual documents the source snapshot in `/home/punam/Music/IMPFolder/Project/120r-copied`. It is intended for someone starting with no knowledge of this repository, an operator preparing a long training run, and a developer maintaining the implementation. Commands assume a Bash terminal in the project root unless a Colab cell is explicitly identified.

The review covers every Python file in `src/llm120`, the standalone tokenizer comparison script, all five test files, all six YAML configurations, packaging metadata, the lockfile, both README files, the Colab dependency file, and every notebook cell. The Colab source copy was compared byte-for-byte with the main source: all shared Python files are identical; only the main tree contains `generatecopy.py`. Generated reference appendices list commands, symbols, settings, dependencies, notebook cells, and source fingerprints.

Evidence labels used throughout:

- **Implemented:** directly established by the inspected source, configuration, or notebook.
- **Repository claim:** a statement in an existing README without a corresponding measurement artifact in this snapshot.
- **Review finding:** a limitation or likely failure inferred from the source; these are not all runtime-reproduced bugs.
- **Recommended procedure:** operational guidance or an improvement proposal, not a feature already implemented.

The inspected folder contains no raw corpus, trained tokenizer, model checkpoints, training metrics, virtual environment, or completed experiment report. Model quality, end-to-end compatibility, GPU fit, and reported throughput cannot be established from this copy. The reviewer did not install the large ML environment, download datasets, or run model training. The dependency-free schedule test passed; the complete test suite failed during collection because the available Python environment lacks required ML packages. See Chapter 16 for the exact validation boundary.

Read Chapters 2-5 for orientation and setup, Chapters 6-10 for base training, Chapters 11-12 for assistant tuning, and Chapters 13-18 for Colab, troubleshooting, testing, internals, and extensions. The appendices are the detailed reference. This is a source-snapshot manual, not a certification of current cloud quotas or upstream library compatibility.

## 2. What the project builds

LLM120 is a single-GPU language-model training harness. It learns a new byte-level tokenizer, constructs a randomly initialized decoder-only Transformer, and trains it to predict the next token in English-oriented text. A later pipeline teaches instruction following and preferences. The public interface is a collection of command-line programs; there is no web application, API server, database, retrieval system, or multi-user deployment layer.

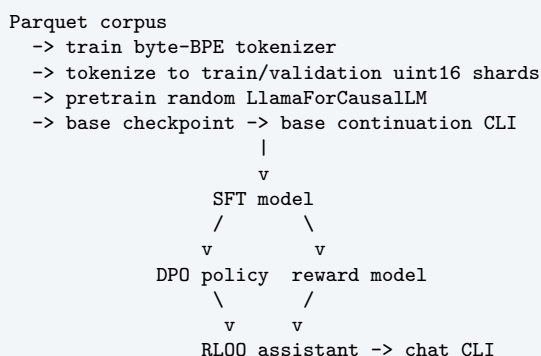
The architecture is instantiated with Transformers' `LlamaConfig` and `LlamaForCausalLM`. The project implements the surrounding training and data machinery, rather than implementing each attention kernel. The default architecture is inspired by the small SmolLM family but does not load pretrained SmolLM or Llama weights for the initial run.

### 2.1 Concepts from zero

| Term       | Meaning in this repository                                                            |
|------------|---------------------------------------------------------------------------------------|
| Token      | A numeric text fragment produced by the tokenizer; not necessarily a whole word.      |
| Vocabulary | The set of token IDs the model can represent; normally 32,768 entries here.           |
| Parameter  | A learned number in the model's embeddings, attention, MLPs, or normalization layers. |
| Context    | One input sequence; the configured training sequence length is 2,048 tokens.          |
| Base model | A model trained on document continuation; it has not yet learned the chat interface.  |
| SFT        | Supervised fine-tuning on conversations, with loss on assistant turns.                |

| Term         | Meaning in this repository                                                                       |
|--------------|--------------------------------------------------------------------------------------------------|
| DPO          | Offline preference optimization using chosen/rejected responses.                                 |
| Reward model | A separate model that assigns scalar preference scores to responses.                             |
| RL00         | Online reinforcement learning using sampled completions and a leave-one-out reward baseline.     |
| Microbatch   | Sequences processed together in one forward/backward pass.                                       |
| Accumulation | Several backward passes before one optimizer update.                                             |
| Checkpoint   | Saved weights and, for resumable training, optimizer/scaler/RNG state.                           |
| Perplexity   | Exponential of average token loss; meaningful only under comparable tokenization and evaluation. |

## 2.2 End-to-end artifact flow



The reward model starts from the SFT checkpoint, not from the DPO checkpoint. RL00 starts its policy from DPO and consumes the separately trained reward model. Each stage writes to a different default directory. DPO can be a useful stopping point; running RLHF is not proof of improvement.

## 2.3 What success means

The first milestone is a reproducible 20-step systems pilot. The second is a base model whose held-out loss improves and whose continuations are locally coherent. The third is an assistant that follows simple instructions under its exact chat template. No successful run is included in this snapshot. Parameter count and a completed token budget do not establish factual accuracy or reasoning ability.

## 3. Repository layout and software environment

|                               |                                       |
|-------------------------------|---------------------------------------|
| 120r-copied/                  |                                       |
| README.md                     | existing overview and recipes         |
| pyproject.toml                | dependencies and 14 console commands  |
| uv.lock                       | resolved package metadata             |
| configs/                      | pilot and five production model sizes |
| src/llm120/                   | main Python package                   |
| tests/                        | ten test functions in five files      |
| scripts/compare_tokenizers.py |                                       |
| colab/                        |                                       |
| LLM120_Colab.ipynb            | notebook orchestration                |
| requirements-colab.txt        | dependencies excluding torch          |
| src/llm120/                   | duplicate package source              |
| docs/                         | this manual and generated references  |

Expected runtime directories, created by commands rather than included in this copy:

|                          |                                |
|--------------------------|--------------------------------|
| data/raw/                | downloaded pretraining Parquet |
| artifacts/tokenizer-32k/ | tokenizer assets and metadata  |
| data/tokenized/          | binary shards and manifests    |
| data/posttrain/raw/      | downloaded assistant datasets  |

|                               |                                   |
|-------------------------------|-----------------------------------|
| data/posttrain/normalized/    | normalized stage-specific Parquet |
| checkpoints/llm120-english/   | base-model run                    |
| checkpoints/llm120-sft/       | supervised run                    |
| checkpoints/llm120-dpo/       | preference run                    |
| checkpoints/llm120-reward/    | reward-model run                  |
| checkpoints/llm120-assistant/ | RL00 run                          |

### 3.1 Dependencies and installation semantics

Python support is  $\geq 3.11, < 3.13$ . Core dependencies include PyTorch, Transformers, Tokenizers, Hugging Face Hub, NumPy, PyArrow, PyYAML, Safetensors, and TensorBoard. The `dev` extra installs `pytest` and `Ruff`; the `posttrain` extra installs `Accelerate`, `Datasets`, and `TRL`. Hatchling builds the package. Console commands map to `main()` functions in package modules.

The repository explicitly routes `torch` to the PyTorch `cu128` index through `uv` configuration. This is the author's selected local GPU environment, not a portable automatic hardware detector. The lockfile records 91 package entries, including platform/version alternatives. Those entries are the recorded resolution, not packages verified to exist in the reviewer's interpreter. A requirements range in `pyproject.toml` is also not a compatibility guarantee across every allowed release.

### 3.2 Local setup

Prerequisites are a supported Python interpreter, `uv`, access to the package indexes and Hugging Face, suitable NVIDIA drivers, a CUDA-capable PyTorch build, and sufficient storage. Verify these on the target machine before spending time on a corpus download.

```
cd /home/punam/Music/IMPFolder/Project/120r-copied
python3 --version
uv --version
nvidia-smi
df -h .
uv sync --locked --extra dev --extra posttrain
source .venv/bin/activate
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
llm120-benchmark --vocab-size 32768 --steps 10
```

`--locked` asks `uv` to preserve the checked-in resolution and fail if it needs updating. If it fails, inspect the dependency conflict or unavailable package before changing the lockfile. Record any revised environment with the run. The installation and benchmark commands above are procedures, not commands executed during this documentation review.

The synthetic-vocabulary benchmark can run before data and tokenizer preparation. `llm120-doctor` cannot pass that early because it expects tokenizer and corpus files. Training requires CUDA even for `precision:fp32`; generation and chat have CPU fallbacks.

### 3.3 Path rules

Most CLI-relative paths are interpreted against the shell's working directory. YAML paths are different: `load_config()` resolves relative paths against the YAML file's grandparent directory. Therefore place a new local configuration in `configs/` or use absolute paths. Moving a YAML file to an arbitrary directory can silently point it at different data or outputs. The Colab notebook uses absolute paths to avoid this problem.

## 4. Architecture and resource planning

### 4.1 Decoder architecture

The model uses causal self-attention, grouped-query attention, rotary positional embeddings, RMS normalization, a SwiGLU-style MLP via the Llama implementation, and tied input/output embeddings. Attention and MLP biases are disabled. `use_cache` is disabled for training. The backend is set to PyTorch SDPA attention. Model weights are initially FP32; mixed precision changes compute precision rather than replacing all master weights with BF16.

For vocabulary  $V$ , width  $H$ , MLP width  $I$ , layers  $L$ , query heads  $Q$ , and key/value heads  $K$ , the tied, bias-free parameter count is:

$$P = V \cdot H + L \cdot (2 \cdot H \cdot H + 2 \cdot H \cdot H \cdot K / Q + 3 \cdot H \cdot I + 2 \cdot H) + H$$

This covers the tied embedding, attention projections, three MLP projections, two per-layer normalization vectors, and final normalization vector. Counts below assume  $V = 32,768$ ; a tokenizer with a different actual length changes the count.

| Configuration | Parameters  | Layers | Width / MLP | Q / KV heads | Micro / accumulation |
|---------------|-------------|--------|-------------|--------------|----------------------|
| pilot, 125m   | 125,077,824 | 30     | 576 / 1536  | 9 / 3        | 4 / 8                |
| 300m          | 299,983,200 | 33     | 800 / 2816  | 10 / 2       | 1 / 32               |
| 350m          | 349,928,800 | 49     | 800 / 2112  | 10 / 2       | 1 / 32               |
| 370m          | 369,987,904 | 44     | 832 / 2496  | 8 / 1        | 1 / 32               |
| 400m          | 404,827,200 | 31     | 960 / 3328  | 12 / 4       | 1 / 32               |

The 370M configuration has one KV head, a multi-query special case. The models are not simple width-scaled versions of each other: their depth, head dimensions, and MLP shapes differ. Throughput and memory must be measured separately.

## 4.2 Token and time arithmetic

```
tokens_per_step = sequence_length * micro_batch_size * accumulation
                  = 2048 * 4 * 8 = 65,536      (default 125M)
max_steps = ceil(max_tokens / tokens_per_step)
            = 152,588                        (10B budget)
actual_processed_tokens = 152,588 * 65,536 = 10,000,007,168
wall_days = token_budget / (tokens_per_second * utilization * 86,400)
```

The slight budget overshoot comes from rounding to complete optimizer steps. The model predicts within fixed blocks, so the counter is input tokens processed, not exactly the number of loss targets. Repeated epochs still count toward this budget. Ten billion requested tokens need not mean ten billion unique tokens from the newly trained tokenizer.

The README reports 6,460-7,449 tokens/s for the 125M model on the author's RTX 5050 laptop, with about 5.04 GiB peak allocated VRAM. Its 17-20 day estimate at 90% utilization is a repository claim, not a measurement reproduced here. The benchmark uses synthetic data and a different update cadence from the accumulated trainer; actual training, validation, checkpoint I/O, and thermal behavior determine real wall time.

## 4.3 Memory and checkpoint storage

An approximate persistent training-state budget is 16 bytes per parameter: FP32 weights, FP32 gradients, and two FP32 Adam moments. Activations, attention workspaces, temporary tensors, CUDA allocations, and library overhead are additional. Approximate serialized checkpoint state is 12 bytes per parameter for weights plus Adam moments, with metadata and other state adding overhead.

| Model | Approx. weight + gradient + Adam bytes | Approx. checkpoint bytes |
|-------|----------------------------------------|--------------------------|
| 125M  | 2.00 GB                                | 1.50 GB                  |
| 300M  | 4.80 GB                                | 3.60 GB                  |
| 400M  | 6.48 GB                                | 4.86 GB                  |

GB here means decimal gigabytes; GiB means bytes divided by  $2^{30}$ . These are arithmetic estimates, not observed allocations. Multiple policy/reference/reward models make post-training a different memory workload from base pretraining. A 400M pretraining configuration does not establish that 400M RLOO fits an 8 GB device.

Checkpoint rotation saves the new checkpoint before deleting old ones. Budget for `keep_last+1` checkpoints at save time, not just the retained count. With two retained 400M checkpoints, transient checkpoint storage is about 14.6 GB before tokenized data and other files. This contradicts the notebook's implication that a 1B/400M run with two retained checkpoints comfortably fits a 15 GB Drive allowance.

The downloader requires free space of at least three times the selected compressed bytes. This may be stricter than the README's final-footprint estimate. The doctor checks only a 20 GiB free-space threshold; passing it is not a complete storage budget.

## 5. First end-to-end pilot

The pilot exercises the real architecture and training loop over 20 optimizer steps. It is not a smaller architecture and is not expected to produce meaningful English.

```
llm120-download --max-shards 4 --output data/raw/pilot
llm120-tokenizer --input data/raw/pilot \
  --output artifacts/tokenizer-pilot --max-documents 100000
llm120-prepare --input data/raw/pilot \
  --tokenizer artifacts/tokenizer-pilot \
  --output data/tokenized-pilot --workers 6
llm120-doctor --config configs/pilot.yaml
llm120-benchmark --config configs/pilot.yaml --steps 10
llm120-train --config configs/pilot.yaml --resume none
llm120-generate --output-dir checkpoints/llm120-pilot \
  --repetition-penalty 1 --no-repeat-ngram-size 0 \
  "The Earth revolves around"
```

Expected artifacts are a download manifest, tokenizer files, binary shards and two manifests, run metadata, TensorBoard events, metrics JSONL, and checkpoints at steps 10 and 20. Verify finite training loss and both scheduled validation events. Output quality is not an acceptance criterion for this systems test.

If rerunning a completed pilot, use `--resumeauto` to inspect the completed run, or copy the pilot YAML to a new file under `configs/` and change its output directory before starting fresh. `--resumenone` refuses to start in a directory containing recognized checkpoints. Do not delete useful checkpoints merely to repeat a smoke test.

The pilot validation subset must contain at least one full 2,048-token block. The shuffled training corpus needs at least two complete blocks because of a reviewed one-block edge case in `data.py`. The default four-shard sample should be much larger, but custom tiny datasets can trigger these constraints.

## 6. Downloading and preparing the corpus

### 6.1 Download behavior

The default repository ID is `EleutherAI/SmolLM2-135M-10B`. `download.py` queries its requested revision, resolves a commit SHA, chooses sorted Parquet files, checks free space, and downloads selected files at that SHA. The output manifest stores the requested and resolved revisions, file names, and compressed size.

```
llm120-download --output data/raw/smollm2-135m-10b
```

For a reproducible later download, supply the earlier manifest's resolved commit with `--revision`. Without that, each invocation resolves the then-current `main`. Existing download directories are not cleaned automatically. Use a clean directory when changing datasets or subsets: downstream preprocessing recursively reads all Parquet files, not just the names in `download-manifest.json`.

`--max-shards` selects evenly spaced indices including endpoints when more than one shard is requested. One shard chooses the middle entry. This is a storage-limited subset, not a verified source-balanced mixture. `--pattern` is only implemented as a prefix before the first `*`, plus a `.parquet` suffix test; it is not full glob matching. Zero passed to `--max-shards` is falsy and selects all files instead of rejecting the value.

### 6.2 Tokenizer training

```
llm120-tokenizer --input data/raw/smollm2-135m-10b \
```

```
--output artifacts/tokenizer-32k --max-documents 500000
```

The tokenizer uses NFC Unicode normalization, byte-level pretokenization without an added prefix space, BPE with minimum frequency two, and the full byte alphabet. Training samples valid strings of at least 64 stripped characters from a quota near the beginning of each shuffled file. Each document is capped at 100,000 characters by default. Small corpora may be reread to reach the requested document count; `training_documents` is the requested count, not a unique-document count.

Special tokens are registered in this order:

```
<|beginoftext|>
<|endoftext|>
<|padding|>
<|unknown|>
<|system|>
<|user|>
<|assistant|>
```

The tokenizer postprocessor itself does not automatically surround text with BOS/EOS. Preprocessing adds them explicitly. The chat template separately formats system, user, and assistant turns; its generation block marks assistant content and EOS for SFT masking. Unsupported roles raise an error. There is no tool-call role or structured tool-use protocol.

The tokenizer is saved in Transformers format with `tokenizer.json`, supporting configuration, and `training-metadata.json`. English, code, and Nepali probes check round-trip reconstruction after NFC normalization. Byte coverage permits Nepali encoding, but does not demonstrate efficient Nepali tokenization or language competence. NFC means round-trip equality to normalized text, not always byte-for-byte equality with arbitrary original Unicode.

## 6.3 Binary preparation

```
llm120-prepare --input data/raw/smollm2-135m-10b \
  --tokenizer artifacts/tokenizer-32k \
  --output data/tokenized --workers 8
llm120-doctor
```

Each spawned worker loads one tokenizer, disables tokenizer-level parallelism, and streams Parquet batches. Non-string or short rows are skipped. Each retained document becomes BOS + encoded text + EOS. Very long texts are encoded in character chunks of up to one million characters. Chunk boundaries can change tokenization near joins because BPE is run separately on each chunk.

The split hash uses the first 16,384 characters plus the original string length, encoded with BLAKE2b. Values below the default 0.001 fraction enter validation. Identical raw documents consistently choose the same split; this is not deduplication, near-duplicate removal, or an exhaustive contamination check. Normalization-equivalent but raw-different strings are not guaranteed to share a split. Tokenizer training is not filtered to exclude model-validation documents.

Workers write little-endian unsigned 16-bit IDs, requiring vocabulary size at most 65,535 under this implementation. Temporary binary files are flushed, fsynced, and replaced. Per-source metadata identifies cached work by source absolute path, size, modification time, tokenizer hash, format version, validation fraction, and minimum characters. Binary sizes must match cached token counts. The fingerprint omits `max_chunk_chars`, so changing that option alone does not invalidate cached output.

Manifest fields are `format_version`, `dtype`, `byte_order`, `split`, `tokenizer_sha256`, `total_tokens`, and `shards`. Each shard records a relative binary path, token count, and source path. Binary checksums are not recorded. The manifests and per-source metadata are not all written with the same atomic replacement procedure as the binary files.

## 6.4 Reading tokens during training

`TokenBlockCorpus` calculates `floor(shard_tokens/sequence_length)` blocks independently for every shard. A shard's final incomplete block is unused; tokens are not carried into the next shard. Documents may span blocks, and multiple documents may occur in one block. BOS/EOS boundaries are present, but there is no per-document attention mask to prevent attention across documents in a block.

Binary files are opened lazily with NumPy memory mapping. File sizes are checked before mapping. Selected blocks are copied into int64 tensors and stacked into batches. An affine permutation  $(a*i+b)\bmod N$ , with  $a$  coprime to  $N$ , traverses all blocks once per epoch. The random seed depends on the epoch, so ordering is reproducible without storing a giant permutation. It is structured shuffling, not a uniformly sampled permutation from all possible permutations.

## 7. Configuration reference and experiment design

There are four YAML sections: `project`, `paths`, `model`, and `training`. The appendices include every configuration verbatim and the exact post-training constructor settings. Dataclasses are frozen; changing a run requires loading a new configuration rather than mutating the object.

### 7.1 Settings that control identity

`project.name` labels the run, `project.output_dir` controls artifacts, and `project.seed` seeds Python, NumPy, PyTorch, CUDA, and the block permutation. `paths.tokenizer`, `paths.train_manifest`, and `paths.validation_manifest` identify the prepared inputs. Architecture fields must match the tokenizer and checkpoint. A change in raw YAML bytes changes the configuration hash, even if only whitespace or a comment changed.

### 7.2 Training settings

| Group            | Fields and behavior                                                                                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Batch and budget | <code>sequence_length</code> , <code>micro_batch_size</code> , <code>gradient_accumulation_steps</code> , <code>max_tokens</code> determine tokens per update and rounded maximum steps. |
| Learning rate    | <code>learning_rate</code> , <code>warmup_steps</code> , <code>decay_ratio</code> , <code>min_learning_rate_ratio</code> define warmup, stable region, and final linear decay.           |
| Optimizer        | <code>adam_beta1</code> , <code>adam_beta2</code> , <code>adam_epsilon</code> , <code>weight_decay</code> , <code>max_grad_norm</code> , <code>fused_optimizer</code> .                  |
| Compute          | <code>precision</code> , <code>gradient_checkpointing</code> , <code>torch_compile</code> , <code>allow_tf32</code> .                                                                    |
| Observability    | <code>log_every_steps</code> , <code>eval_every_steps</code> , <code>eval_batches</code> .                                                                                               |
| Persistence      | <code>save_every_steps</code> , <code>keep_last_checkpoints</code> .                                                                                                                     |

The production defaults use 2,000 warmup steps, a final 20% decay region, minimum LR ratio zero, Adam betas 0.9/0.95, epsilon 1e-8, weight decay 0.01, and gradient norm limit 1.0. The 125M peak LR is 0.003; larger configurations use 0.0015. Weight decay excludes embeddings and parameters with fewer than two dimensions.

Validation checks head divisibility, context length, precision choices, decay ratio, and warmup shorter than the calculated run. It does not validate all positivity constraints. Use positive batch sizes, accumulation, sequence lengths, save/eval/log intervals, and retention counts. Zero intervals can cause modulo errors. A negative or zero budget is not a supported operating procedure.

Changing microbatch and accumulation while preserving their product keeps effective batch size constant, but still changes the YAML hash and prevents exact resume. Make such changes before a run, or deliberately start a separately identified experiment. `--max-steps` changes the training horizon without changing the YAML hash, including the schedule horizon; it should be treated as a smoke-test override rather than a transparent way to extend a production run.

## 8. Pretraining internals and execution

### 8.1 Starting a run

```
llm120-benchmark --config configs/train_125m.yaml --steps 20
llm120-train --config configs/train_125m.yaml --resume none
```

For a separate larger run:

```
llm120-benchmark --config configs/train_400m.yaml --steps 20
llm120-train --config configs/train_400m.yaml --resume none
```

The larger run reuses the production tokenizer and corpus, but writes to `checkpoints/llm120-english-400m`. Pass that path explicitly to downstream SFT; the default SFT input still selects the 125M run.

## 8.2 One optimizer step

The trainer initializes CUDA precision flags and seeds, opens both corpora, chooses a checkpoint or random initialization, enables configured activation checkpointing, places the model on CUDA, and constructs AdamW. FP16 uses a GradScaler; BF16 and FP32 use a disabled scaler interface.

For zero-based optimizer step  $s$ , the first data sample is  $s \times \text{accumulation} \times \text{micro\_batch\_size}$ . Each accumulation pass fetches its deterministic block range, transfers pinned memory to CUDA, computes causal language-model loss using `labels=input_ids`, divides loss by accumulation, checks for finite loss, and backpropagates. After all passes it unscales gradients, clips their norm, computes the schedule factor, updates the optimizer, updates the scaler, and clears gradients.

The Transformers causal-LM implementation shifts labels internally. There are no padding positions in the fixed-block pretraining input. No explicit masking is applied to BOS, EOS, or document boundaries. In FP16, GradScaler may skip an optimizer update after detecting invalid gradients; this loop still advances its step and processed-token counters. Finite loss alone does not prove every update changed weights.

## 8.3 Learning-rate schedule

Warmup factor is  $(\text{step}+1)/\text{warmup\_steps}$ , bounded below by the minimum ratio. After warmup the factor remains one until the final decay interval. It then decreases linearly to the minimum ratio. With minimum ratio zero, the final scheduled update uses learning rate zero. `--max-steps` can shorten the horizon without lowering the YAML's warmup setting, so use the dedicated pilot configuration for meaningful short-run schedule checks.

## 8.4 Evaluation and logging

Training metrics append to `metrics.jsonl`: step, processed tokens, interval mean loss, learning rate, gradient norm, tokens/s, and peak allocated VRAM. TensorBoard receives `train/*` metrics plus `validation/loss` and `validation/perplexity`. Validation is printed to the terminal but is not added to the same JSONL stream.

Validation reads the first configured number of batches from the unshuffled corpus. It is a fixed subset unless large enough to cover the corpus. A last partial batch wraps to earlier blocks because corpus access is cyclic. Reported perplexity is  $\exp(\min(\text{loss}, 20))$ , with an explicit numerical cap. It is not an independent downstream task benchmark.

```
tensorboard --logdir checkpoints/llm120-english/tensorboard
```

Training throughput intervals can include time spent in earlier evaluation/checkpoint work. Benchmark throughput excludes its first two iterations, uses synthetic random tokens, performs an optimizer update every microbatch, always requests fused AdamW, and does not use FP16 GradScaler. It is useful for comparing capacity and approximate throughput, not for proving stable training under the exact production loop.

# 9. Checkpoints, stopping, and reproducibility

## 9.1 Base checkpoint structure

```
checkpoints/llm120-english/
  run.json
  metrics.jsonl
  tensorboard/
  checkpoint-000001000/
    model/                Transformers weights and tokenizer
    trainer-state.pt      optimizer, scaler, RNG, step, metadata
```

|               |                                      |
|---------------|--------------------------------------|
| metadata.json | identities and processed-token count |
|---------------|--------------------------------------|

The checkpoint is assembled in a hidden temporary directory and renamed into place. Only afterward are old numbered checkpoints deleted. Discovery sorts the numeric suffix. The default keeps three checkpoints; the pilot keeps two. Atomic directory rename on a local filesystem reduces exposure to partially written checkpoints, but does not establish crash durability for every filesystem or cloud-mounted Drive.

## 9.2 Graceful stopping and resuming

Ctrl-C and SIGTERM set a stop flag. The current accumulation/update completes, scheduled work may run, and a checkpoint is written before exit. Abrupt process termination, power loss, disk-full errors, and non-finite-loss exceptions do not follow this graceful path; the last successful checkpoint is the recovery point.

```
llm120-train --config configs/train_125m.yaml --resume auto
llm120-train --config configs/train_125m.yaml \
  --resume checkpoints/llm120-english/checkpoint-000001000
```

The explicit checkpoint above is an example only; select an existing directory. `auto` chooses the highest-numbered checkpoint, not the best validation score. Resuming an older checkpoint inside an output directory with later checkpoints can create conflicting step names and ambiguous logs. Use the latest valid checkpoint for routine continuation.

## 9.3 What identity checks cover

Base metadata records SHA-256 for the YAML, tokenizer JSON, train manifest, and validation manifest. Resume compares the first three; the recorded validation-manifest hash is not checked. It restores optimizer, scaler, and Python/NumPy/Torch/CUDA RNG states. The data position is derived from completed step and batching settings.

Binary shard contents are not hashed, and source-code or library versions are not enforced. Changed bytes of unchanged size can evade integrity checks. The trainer does not automatically run the doctor's tokenizer-versus-manifest validation before a fresh run. Run the doctor after preparing or copying data. Reproducible positioning does not guarantee bitwise identical floating-point output across hardware, drivers, kernels, or dependency versions.

Keep a separate experiment record containing source fingerprints, config, lockfile, GPU/driver details, download revisions, tokenizer identity, manifests, validation prompts, and results. Only load trusted `trainer-state.pt` files: the loader explicitly uses `torch.load(..., weights_only=False)` to restore Python state.

# 10. Evaluating and generating from the base model

Use document prefixes for initial evaluation. A base model has been taught continuation, not necessarily question-answer formatting.

```
llm120-generate --repetition-penalty 1 --no-repeat-ngram-size 0 \
  "Photosynthesis is the process by which"
llm120-generate --sample --temperature 0.8 --top-p 0.95 \
  "In a small village near the mountains,"
```

The first command explicitly disables repetition suppression. This matters because the current `generate.py` defaults are penalty 1.1 and no-repeat n-gram size 4, despite the README describing unsuppressed generation. The unregistered `generatecopy.py` retains older behavior and is not the implementation behind `llm120-generate`.

Generation resolves the latest numbered base checkpoint unless `--checkpoint` is supplied. It supports a checkpoint's nested `model/` directory or a direct Transformers model directory. It prepends BOS, generates up to 128 new tokens by default, and prints the original prompt plus continuation with special tokens removed. It uses BF16 on supporting CUDA hardware and FP32 otherwise, including on CPU. It does not explicitly enable KV caching after loading the saved training configuration.

The input is not explicitly truncated or checked against the context limit. Keep prompt length plus requested completion length within the evaluated context range. Fix the seed and decoding settings for comparisons. A

good sampled example is not a substitute for held-out evaluation, and repetition suppression can hide failure modes.

Before assistant tuning, inspect loss curves, several fixed greedy continuations, repeated-output behavior, and fresh held-out examples. A downstream benchmark harness is not included. If adding one, record the task version, split, tokenization, prompt format, and exact checkpoint. Do not compare perplexity across different tokenizers as if it were a directly equivalent score.

## 11. Assistant data and normalization

### 11.1 Sources and files

| Stage  | Configured dataset                                  | Normalized representation                                            |
|--------|-----------------------------------------------------|----------------------------------------------------------------------|
| SFT    | HuggingFaceTB/smol-smoltalk                         | <b>messages</b> conversation list                                    |
| DPO    | argilla/ultrafeedback-binarized-preferences-cleaned | shared <b>prompt</b> , <b>chosen</b> , <b>rejected</b> message lists |
| Reward | nvidia/HelpSteer2 preference JSONL                  | prompt, chosen/rejected assistant turns, strength                    |
| RLOO   | Prompts extracted from normalized reward data       | <b>prompt</b> only                                                   |

```
llm120-download-posttrain
llm120-prepare-posttrain
```

The downloader resolves each repository to a commit SHA per invocation and records it in `data/posttrain/raw/download-manifest.json`. `--onlysft`, `--onlydpo`, or `--onlyreward` selects one download. The preparer still expects all three directories; it has no `--only` option. There is no CLI revision selector for post-training downloads, so preserving an earlier snapshot and manifest matters for reproducibility.

Normalized output contains eight Parquet files, one train and one validation file for each of `sft`, `dpo`, `reward`, and `rlhf`, plus `manifest.json` with counts, seed, requested validation rows, and system prompt. Normalization does not record upstream revision hashes in that output manifest; retain the raw download manifest alongside it.

### 11.2 SFT normalization

SFT discovery classifies files by words in their file names, not directory names. It uses existing validation/test/eval files when detected; otherwise it makes a seeded row split with approximately 1% validation capped at 2,000 rows. Valid conversations have at least two messages, recognized roles, non-empty string content, and a final assistant message. Strict role alternation and a preceding user turn are not enforced.

If no train-named file is detected, the fallback uses all files as training input. If separate validation files were also detected, they may then overlap with training. Check actual source file names and split counts before trusting this heuristic with a new corpus.

### 11.3 DPO normalization

The loader merges discovered Parquet files, checks chosen/rejected columns, validates message lists, and removes their longest shared message prefix into an explicit prompt. Empty remaining completions are filtered. A pair with no shared prefix raises an exception during mapping instead of being skipped. The result is split by rows at approximately 1%, capped at 2,000.

This process does not deduplicate prompts across train/validation or guarantee preservation of upstream splits. If multiple pairs share a prompt, leakage is possible. The code's cleaning is schema preparation; dataset-wide contamination removal is not implemented here.

### 11.4 HelpSteer and RL prompts

The preparer finds a non-empty preference JSONL file outside `.cache` paths. Zero-strength ties are removed. Negative strength chooses response 1; positive strength chooses response 2. Both `response_1/response_2` and

**response1/response2** naming variants are accepted. A fixed system message and user prompt form the shared conversation prefix.

Existing **train** versus **validation/test** labels are used when both sides are populated; otherwise a seeded row split holds out roughly 10%, capped at 2,000 rows. Reward strength is retained but no explicit strength-based weighting is configured in the reward trainer.

RL prompts contain no chosen answer. Prompt deduplication happens independently inside each split, not globally across splits. Raw response filtering converts values to strings for emptiness checks, so null-like values may pass before later processing. Inspect normalized examples and shared-prompt overlap rather than assuming full validation from the README's wording.

## 12. Assistant training stages

All four stage programs use the TRL/Transformers Trainer stack and require BF16-capable CUDA in **require\_cuda()**. They use FP32-loaded trainable weights, fused AdamW, TensorBoard reporting, seed 42, cosine schedules, and approximately 10% warmup. Most batch, precision, and schedule choices are hardcoded in the stage module rather than configurable by the base YAML.

| Setting                  | SFT            | DPO            | Reward          | RLOO           |
|--------------------------|----------------|----------------|-----------------|----------------|
| Default input            | base           | SFT            | SFT             | DPO + reward   |
| LR                       | 3e-4           | 1e-6           | 2e-5            | 1e-6           |
| Micro / accumulation     | 2 / 8          | 1 / 16         | 2 / 8           | 2 / 8          |
| Duration                 | 2 epochs       | 2 epochs       | 2 epochs        | 250 steps      |
| Length bound             | 2048 total     | 1024 total     | 1024 total      | 128 completion |
| Activation checkpointing | disabled       | enabled        | enabled         | enabled        |
| Evaluate / save every    | 500            | 500            | 250             | 50             |
| Evaluation subset        | first 1000 max | first 1000 max | full validation | first 128 max  |

The SFT setting is a verified discrepancy from the README's claim that all trainers use activation checkpointing. Effective RLOO prompt/completion grouping additionally depends on TRL's implementation; do not interpret its batch product as directly equivalent to the supervised token batch.

### 12.1 SFT

```
llm120-sft
llm120-chat --model checkpoints/llm120-sft --greedy \
  "Explain why the sky is blue."
```

SFT trains only on assistant spans using the tokenizer's generation-marked chat template. Packing is disabled. The trainer uses up to 12 dataset preprocessing processes and four data-loader workers. These CPU defaults may be excessive on smaller machines. There is no CLI flag for changing them or enabling SFT activation checkpointing; an intentional source/configuration refactor is needed.

For a larger base model, provide its path explicitly:

```
llm120-sft --model checkpoints/llm120-english-400m \
  --output checkpoints/llm120-sft-400m
```

### 12.2 DPO

```
llm120-dpo
llm120-chat --model checkpoints/llm120-dpo --greedy \
  "Give three ways to save water at home."
```

DPO compares chosen and rejected responses with a sigmoid preference objective, beta 0.5, and reference log-probability precomputation in batches of two. It is offline preference learning, not reward-model-based online

RLHF. Evaluate whether the output improves over the same SFT checkpoint under identical prompts and decoding.

## 12.3 Reward model

```
llm120-reward
```

The reward stage loads the SFT backbone as a sequence-classification model with one scalar label; its classification head must be trained. Pairwise preference training is configured through `RewardTrainer`, with a reward-centering coefficient of 0.01. Evaluation metrics are saved under the final artifact. RLOO expects `eval_accuracy` there.

## 12.4 RLOO

```
llm120-rlhf
llm120-chat "What causes rain? Explain simply."
```

The gate requires finite held-out reward-model accuracy of at least 0.55. `--allow-weak-reward-model` bypasses that numerical gate but does not create a missing metrics file. The 55% threshold is a minimal configured gate, not a general proof of reward-model quality.

RLOO samples two generations per prompt, up to 128 completion tokens, temperature 0.8, top-p 0.95, repetition penalty 1.05, beta 0.1, one iteration, epsilon 0.2, normalized advantages, and truncated-completion masking. Reward functions are the learned model and a small completion-health penalty, weighted 1.0 and 0.1 respectively.

The auxiliary reward penalizes repeated word trigrams and outputs shorter than two words. It does not measure truth, usefulness, or reasoning. Although the function receives completion token IDs, it does not use them beyond pairing lengths. The frozen-reference/KL and online-update mechanics are delegated to the installed TRL implementation; this repository does not reimplement the algorithm.

Compare logged completions, KL-related metrics, length/truncation, repetition, and fixed-prompt quality against the starting DPO model. Higher learned reward alone can reflect reward exploitation. The short default is an experiment, not a promise of better answers.

## 12.5 Outputs, smoke runs, and resume differences

TRL checkpoints use the library's own layout. A completed stage additionally writes `final/` containing the Transformers model, tokenizer, and `stage-metrics.json`. The final directory is an inference artifact; do not assume it carries full resumable optimizer state. `--resumeauto` uses Trainer checkpoints, not the base trainer's custom `trainer-state.pt`.

```
llm120-sft --max-steps 20 --resume none \
--output checkpoints/sft-smoke
```

This example requires a usable base checkpoint and normalized SFT data. A 20-step stage generally finishes before its periodic checkpoint interval; a final model is saved, but a mid-stage interruption may leave no resumable checkpoint. Unlike base training, post-training resume does not enforce project-specific config/tokenizer/data hashes.

Model resolution prefers `final/`, then a model at the supplied directory, then lexicographically sorted checkpoint names. Lexicographic ordering can select an older non-zero-padded Trainer checkpoint. Use an explicit checkpoint or known final path when the parent directory contains several unfinished stage checkpoints. An old `final/` can also take precedence over newer checkpoints.

## 12.6 Chat behavior

`llm120-chat` is a single-prompt command, not an interactive conversation loop. It builds a system and user message, applies the saved chat template with an assistant generation prompt, and prints only the generated completion. Sampling is on by default, with temperature 0.7, top-p 0.9, up to 256 new tokens, and a fixed repetition penalty of 1.05. `--greedy` disables sampling but retains that penalty. There is no conversation history persistence, streaming UI, or tool execution.

## 13. Google Colab path

The notebook is a self-contained orchestration layer for base pretraining and generation. Its source copy matches the main package. The post-training modules are present but not wired into the notebook, and its requirements omit post-training dependencies. Current service availability, quotas, and timeouts were not externally verified; numerical service limits in the bundled README are historical assumptions, not guarantees.

### 13.1 Notebook sequence

Upload the complete `colab/` directory, open `LLM120_Colab.ipynb`, choose an available GPU runtime, edit the settings, and execute the setup cells in order. The default is a 400M model, 1B processed-token budget, microbatch two, two retained checkpoints, save interval 250, and 300,000 tokenizer-training documents.

The notebook mounts Drive, installs non-torch dependencies, places Drive source on `PYTHONPATH`, and detects BF16 support to select BF16 or FP16. First-session preparation downloads an estimated shard subset, trains or reuses a tokenizer, tokenizes locally, and copies the result to Drive. Later sessions copy cached token binaries to `/content/llm120_data/tokenized` for local reads. Checkpoints and tokenizer files remain on Drive.

Shard count is `round(MAX_TOKENS*1.2/117,000,000)`, clamped to 1-85. It is a corpus-size heuristic, not an exact cap on prepared data. Reusing the cached corpus depends only on the training manifest existing; changing the budget does not rebuild it. Actual tokens depend on tokenizer and data content.

The generated YAML uses sorted keys to stabilize its bytes across sessions. Changing GPU precision between sessions changes the config hash and can block resume. The microbatch calculation uses integer division; arbitrary values that do not divide the target batch can change the actual effective batch. The fixed 2,000 warmup steps also prevents very short token budgets from passing configuration validation.

### 13.2 Blocking issues before relying on Colab

**Doctor incompatibility:** `doctor.py` unconditionally requires compute capability at least 8.0 and BF16 when a CUDA device exists, even for an FP16 configuration. The notebook's intended T4 path has capability 7.5 and no BF16, so its doctor cell exits with failure. The notebook's FP16 selection does not resolve this. A code change making checks precision-aware, followed by validation of the complete FP16 path, is needed for a clean T4 workflow. This manual does not silently patch application code.

**Storage peak:** save-before-prune means two retained 400M checkpoints temporarily require storage for three, in addition to data and Drive contents. Set a retention/storage plan before starting, and measure available capacity and checkpoint sizes. The notebook's free-space query on a mounted filesystem may not accurately represent account quota.

**Session recovery:** a new runtime needs dependency installation, import-path setup, and helper definitions again. Run all setup cells through config/training in order, skipping optional benchmarking and generation. Do not follow the abbreviated README sequence if it omits installation and `PYTHONPATH` setup.

**Operational limits:** browser reconnection JavaScript in the notebook is not part of checkpoint correctness and does not guarantee a persistent runtime. Plan for termination and periodic saves. Drive's mounted filesystem should not be assumed to offer the same durability as a tested local filesystem.

## 14. Tokenizer comparison and multilingual extension

### 14.1 Comparison script

```
python scripts/compare_tokenizers.py \
--tokenizer-a artifacts/tokenizer-a \
--tokenizer-b artifacts/tokenizer-b \
--data data/raw/smollm2-135m-10b \
--output reports/tokenizer-comparison
```

The paths above are illustrative and must refer to existing trained tokenizers. The script samples equal per-shard quotas for English and code, by default 50,000 documents in each domain from rows after 90,000. It recognizes `dclm_edu`, `fineweb_edu`, `cosmopedia_v2`, and `stack_edu` source labels. Input needs both `text` and `source`

columns. It excludes the base-model validation hash slice, NFC-normalizes text, and caps document length at 16,384 characters.

It warms both tokenizers and alternates measurement order per batch. Metrics include total tokens, characters/token, bytes/token, tokens/word, context capacity, long-document fraction, document fertility percentiles, unknown IDs, round-trip failures, observed vocabulary, encoding throughput, and paired document wins. It saves `report.json` and `report.md` and requires `training-metadata.json` in both tokenizer directories.

The term held-out here means a sampling heuristic intended to avoid tokenizer-training prefixes; it is not model validation data, and overlap is not verified against stored training sample identities. Every shard must supply both domain quotas or the script raises. It is therefore not a generic comparison tool for arbitrary Parquet datasets.

## 14.2 Nepali adaptation

Nepali support in the current code is byte coverage and a probe. No Nepali corpus builder, vocabulary expansion, embedding initialization, continuation recipe, or Nepali evaluation suite is implemented. The README proposes preserving all existing token IDs, adding Nepali merges, initializing new embedding rows, and continuing training with English replay. Treat this as a future experiment requiring new code and tests. Replacing the tokenizer under existing token binaries or checkpoints is invalid.

## 15. Troubleshooting and operational checklist

| Symptom                                   | Likely cause / next action                                                                                 |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Command not found                         | Activate the environment and confirm editable/package installation produced console scripts.               |
| Missing torch/TRL/Datasets                | Install the correct extras in the same interpreter that runs the command.                                  |
| CUDA unavailable                          | Inspect driver visibility and the installed torch build before downloading data.                           |
| BF16 failure on T4                        | Base trainer can select FP16, but doctor is not precision-aware; post-training requires BF16.              |
| GPU out of memory                         | Benchmark the intended model; reduce microbatch before a new run; inspect SFT's disabled checkpointing.    |
| Downloader rejects disk capacity          | Its rule is three times compressed bytes; account separately for token binaries and transient checkpoints. |
| No complete blocks                        | Increase usable corpus/split size or deliberately choose a shorter sequence in a new config.               |
| Empty-range error in shuffled tiny corpus | One complete block triggers <code>_permutation()</code> edge case; use at least two pending a code fix.    |
| Tokenizer SHA mismatch                    | Locate the tokenizer used to produce the corpus; do not simply edit hashes to force acceptance.            |
| Resume refused after config edit          | Restore the exact original YAML and inputs or start a new experiment.                                      |
| Non-finite loss                           | Preserve the last valid checkpoint; inspect precision, gradients, learning rate, and inputs.               |
| Checkpoint save disk-full                 | Budget one extra checkpoint during rotation and retain a known-good recovery point.                        |
| Poor output after pilot                   | Expected; the pilot is a systems check, not sufficient language training.                                  |
| DPO no shared prompt                      | Inspect the offending chosen/rejected schema; the current mapper raises instead of skipping.               |
| RLHF gate fails                           | Inspect <code>final/stage-metrics.json</code> , reward data quality, and held-out accuracy.                |
| Wrong downstream model chosen             | Supply an explicit final/model path; parent checkpoint resolution has ordering pitfalls.                   |
| Colab resume fails on another GPU         | Precision changed the generated YAML hash; maintain a consistent run identity.                             |

Before a long run, complete installation, synthetic benchmark, preparation, doctor, real-data benchmark, and pilot. Confirm the intended output path, data count, parameter count, schedule, free space, and checkpoint retention. Keep the machine powered and storage stable. During the run, inspect loss, gradients, validation,

throughput, thermal behavior, and successful saves. Afterward, retain source/environment provenance and a fixed evaluation report with the exported model.

There is no automatic best-checkpoint selection, alerting, distributed training launcher, artifact upload, experiment registry, or production serving layer. These are potential additions rather than hidden features.

## 16. Tests and validation performed for this manual

### 16.1 Included tests

| Test file                      | Assertions represented                                                                                      |
|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>test_data.py</code>      | Complete per-epoch block permutation; identical block retrieval from independent corpus instances.          |
| <code>test_doctor.py</code>    | Manifest tokenizer hash and binary-size checks.                                                             |
| <code>test_download.py</code>  | Evenly spaced subset includes expected endpoints and handles oversized count.                               |
| <code>test_schedule.py</code>  | Warmup, stable interval, monotonic decay, and final zero factor.                                            |
| <code>test_posttrain.py</code> | Shared-prompt extraction, HelpSteer sign, repetition penalty, cache-lock exclusion, and warmup calculation. |

There are ten test functions. The stateless data test does not save or restore a real optimizer checkpoint. None of these tests proves multi-stage GPU training, exact numeric resume, cloud durability, or model quality. No automatic CI configuration is present in the inspected tree.

### 16.2 Review execution results

The following were run with the available Python 3.12 interpreter without installing the ML environment:

```
PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=src \
python3 -m pytest tests/test_schedule.py -q -p no:cacheprovider
# Result: 1 passed

PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=src \
python3 -m pytest -q -p no:cacheprovider
# Result: collection stopped with 4 import errors.
# Missing torch and huggingface_hub were reported.
```

All project Python files and notebook code cells were additionally syntax-checked without importing ML modules; the generated review metadata records counts. Source-copy equivalence and the parameter-count arithmetic were checked. The PDF build is checked separately for successful compilation, extracted text, and representative page rendering. No training benchmark or model-generation result is claimed by these checks.

Recommended next validation after installing the pinned environment is `python-mpytest-q`, followed by the 20-step base pilot, a controlled interruption/resume exercise in a separate run, and small isolated SFT/DPO/reward/RLOO smoke runs after their prerequisites exist. Add meaningful regression tests for the concrete issues in the next chapter before fixing them.

## 17. Review findings and maintenance priorities

These findings describe this snapshot. They were identified by code inspection unless the test section explicitly says otherwise. No application fixes are included in this documentation task.

### 17.1 Address before a constrained or Colab run

- The doctor rejects FP16-only GPUs regardless of selected precision, blocking the intended T4 notebook path.
- Checkpoint rotation needs storage for one more checkpoint than the retained count; the Colab storage estimate omits this peak.

- SFT disables activation checkpointing while the README states the opposite. Measure its memory independently.
- The benchmark differs from training in accumulation, optimizer grouping, fused-optimizer fallback, and FP16 scaling; its numbers are estimates.
- Core ML dependencies are absent from the review environment, so end-to-end package/API compatibility remains unverified.

## 17.2 Reproducibility and integrity gaps

- Base resume records but does not compare the validation-manifest hash.
- Binary token shards are size-checked, not content-hashed; the trainer does not automatically perform every doctor check.
- `max_chunk_chars` is omitted from preprocessing cache identity, although it can change token sequences.
- Tokenizer sampling may reread documents and does not exclude model-validation content; metadata does not identify the exact sample set.
- Post-training resume has no equivalent project-specific identity checks. Its model resolver sorts checkpoint names lexicographically and prioritizes possibly stale final exports.
- Config byte hashing is strict while source, environment, and `--max-steps` horizon changes are not checked; preserve a separate run manifest.
- A new download resolves current repository heads unless explicitly controlled; post-training has no revision CLI option.

## 17.3 Data and edge cases

- A shuffled corpus of exactly one block fails because `randrange(1,1)` is empty.
- Config validation leaves several zero/negative settings unchecked; use validated positive operating values.
- DPO mapping aborts on pairs with no shared prefix. SFT filename heuristics and row-wise preference splitting can produce overlap.
- RL prompt deduplication is within each split, so prompt independence across train/validation is not guaranteed.
- HelpSteer null-like response values are not robustly rejected by string-based non-emptiness checks.
- Downloader pattern matching is narrower than a full glob, and a zero shard count means all files.
- The main generator's repetition defaults disagree with documentation. The old `generatecopy.py` is unregistered duplicate code.
- The main and Colab source copies can drift in future changes; currently they match for every shared file.

## 17.4 Suggested development order

First make hardware checks precision-aware and add the one-block/config/cache-identity regression tests. Next improve run manifests, numeric checkpoint resolution, binary integrity, and split isolation. Then validate full dependency compatibility and automate a small integration pipeline. Finally add model-quality evaluation, artifact packaging, and any serving or multilingual work. These are proposed changes, not additional work silently performed in this review.

## 18. Developer walkthrough and extension boundaries

### 18.1 Module responsibilities

| Module                             | Responsibility and main relationships                                                                        |
|------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>config.py</code>             | Frozen config dataclasses, root-relative path resolution, limited validation, derived batch/step arithmetic. |
| <code>download.py</code>           | Hub revision resolution, subset selection, disk check, pinned snapshot, download manifest.                   |
| <code>tokenizer_train.py</code>    | Text iterator, byte-BPE construction, role template, probes, saved tokenizer metadata.                       |
| <code>prepare.py</code>            | Spawned tokenization workers, split hash, binary writing, cache metadata, split manifests.                   |
| <code>data.py</code>               | Lazy memory maps, per-shard blocks, deterministic affine ordering, batch tensors.                            |
| <code>model.py</code>              | Llama configuration/model construction, parameter counting, decay/no-decay optimizer groups.                 |
| <code>schedule.py</code>           | Pure warmup/stable/linear-decay function.                                                                    |
| <code>checkpoint.py</code>         | Numeric discovery, RNG capture/restore, temporary checkpoint save, rotation, trainer-state load.             |
| <code>train.py</code>              | CUDA setup, initialization/resume, optimizer loop, validation, logging, signals, final checkpoint.           |
| <code>doctor.py</code>             | Hardware, tokenizer hash, manifest size/count, disk checks.                                                  |
| <code>benchmark.py</code>          | Synthetic forward/backward benchmark and rough wall-time estimate.                                           |
| <code>generate.py</code>           | Base continuation loading, decoding arguments, prompt-plus-output printing.                                  |
| <code>posttrain_download.py</code> | Three assistant dataset specifications, revision snapshots, merged raw manifest.                             |
| <code>posttrain_prepare.py</code>  | Conversation checks, chosen/rejected normalization, HelpSteer preference conversion, RL prompts.             |
| <code>posttrain_common.py</code>   | BF16 CUDA gate, model/tokenizer resolution, split loading, warmup, Trainer resume, final exports.            |
| <code>sft.py</code>                | SFTTrainer configuration and assistant-only supervised run.                                                  |
| <code>dpo.py</code>                | DPOTrainer configuration and offline preference run.                                                         |
| <code>reward.py</code>             | Scalar sequence-classification reward model and RewardTrainer run.                                           |
| <code>rlhf.py</code>               | Reward-accuracy gate, completion-health reward, RLOOTrainer run.                                             |
| <code>chat.py</code>               | One-turn chat template application and completion-only output.                                               |
| <code>__init__.py</code>           | Package description and version string.                                                                      |
| <code>generatecopy.py</code>       | Older unregistered base generator; not a second supported console interface.                                 |

The symbol appendix gives names and line locations for every top-level function/class and class method. Python files are deliberately small and role-focused. Entry-point modules mostly define `parse_args()` plus `main()`; internal modules expose reusable helpers.

### 18.2 Adding a model or dataset

For a new architecture size within Llama’s supported shape, copy a YAML inside `configs/`, choose a distinct output directory, verify head divisibility and parameter arithmetic, and benchmark before preparing a major run. Changes beyond this family require extending `model.py` and validating checkpoint loading, training, and generation together.

For a new pretraining dataset, supply Parquet with a `text` column. `source` is optional for the core pipeline but required by the tokenizer comparison script’s domain logic. A repository with a different file layout may require adjusting downloader selection. The project does not perform broad text cleaning, quality classification, or deduplication; add and document those stages explicitly.

For new instruction/preference schemas, normalize into the existing conversational structures and test the shared-prompt and role-template behavior. Add dataset-revision provenance to normalized artifacts. Avoid interpreting successful schema conversion as evidence of license suitability, split independence, or answer quality.

## 18.3 Export and serving boundary

Base inference assets live under a numbered checkpoint's `model/`; assistant inference assets live under a stage's `final/`. Keep the model configuration, weight files, tokenizer files, and chat template together. Resumable training additionally needs the respective trainer-state/checkpoint layout. The code provides local Transformers inference but no quantization exporter, GGUF conversion, ONNX pipeline, HTTP API, Docker deployment, or publication command.

Before sharing a trained artifact, document the actual dataset provenance, experiment budget, known limitations, evaluation results, and usage conditions. No standalone project LICENSE file is present in this snapshot; do not assume a distribution license from the code's mere availability. This is an inventory observation, not a legal determination about upstream sources.

## 18.4 Rebuilding this documentation

The human-written source is `docs/manual.md`. `docs/build_manual.py` reads that text, extracts the reference appendices from the source tree, writes a complete Markdown copy and LaTeX, and compiles the PDF using local TeX tools. It also writes a JSON inventory with hashes and syntax checks. Run it from an environment with Python 3.11+ and `pdflatex` installed; it does not import torch or contact external services.

```
python3 docs/build_manual.py
```

The resulting manual is tied to the source fingerprints in its inventory. Regenerating reference tables does not automatically update the authored explanations after application changes; review both parts together.

## A. Complete command-line reference

The project registers 14 console commands. The following flags and defaults come from the reviewed source, not from executing imports in the unavailable ML environment. Each argparse program also provides `--help`. Optional values with no explicit default normally resolve to None; required options and positionals still require input.

### A. llm120-download

Entrypoint: `llm120.download:main`.

Source: `src/llm120/download.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--repo-id` – default: `DEFAULT_DATASET`.
- `--revision` – default: `main`.
- `--output` – default: `Path('data/raw/smollm2-135m-10b')`; type: `Path`.
- `--pattern` – default: `data/*.parquet`.
- `--max-shards` – type: `int`; no explicit default. Download an evenly spaced subset for a pilot run; omit for the full dataset

### A. llm120-tokenizer

Entrypoint: `llm120.tokenizer_train:main`.

Source: `src/llm120/tokenizer_train.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--input` – default: `Path('data/raw/smollm2-135m-10b')`; type: `Path`.
- `--output` – default: `Path('artifacts/tokenizer-32k')`; type: `Path`.
- `--vocab-size` – default: `32768`; type: `int`.
- `--max-documents` – default: `500000`; type: `int`.
- `--max-chars-per-document` – default: `100000`; type: `int`.
- `--seed` – default: `42`; type: `int`.

### A. llm120-prepare

Entrypoint: `llm120.prepare:main`.

Source: `src/llm120/prepare.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--input` – default: `Path('data/raw/smollm2-135m-10b')`; type: `Path`.
- `--tokenizer` – default: `Path('artifacts/tokenizer-32k')`; type: `Path`.
- `--output` – default: `Path('data/tokenized')`; type: `Path`.
- `--workers` – default: `max(1, min(8, (os.cpu_count() or 4) - 2))`; type: `int`.
- `--validation-fraction` – default: `0.001`; type: `float`.
- `--min-chars` – default: `64`; type: `int`.
- `--max-chunk-chars` – default: `1000000`; type: `int`.

## A. llm120-train

Entrypoint: `llm120.train:main`.

Source: `src/llm120/train.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--config` – default: `Path('configs/train_125m.yaml')`; type: `Path`.
- `--resume` – default: `auto`. `auto`, `none`, or an explicit checkpoint directory
- `--max-steps` – type: `int`; no explicit default. Override max steps (useful for smoke tests)

## A. llm120-generate

Entrypoint: `llm120.generate:main`.

Source: `src/llm120/generate.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `prompt` – default: `The solar system consists of`; nargs: `?`.
- `--checkpoint` – type: `Path`; no explicit default.
- `--output-dir` – default: `Path('checkpoints/llm120-english')`; type: `Path`.
- `--max-new-tokens` – default: `128`; type: `int`.
- `--min-new-tokens` – default: `0`; type: `int`. Require at least this many generated tokens before EOS can stop generation
- `--sample` – action: `store_true`; default: `False`.
- `--temperature` – default: `0.8`; type: `float`.
- `--top-p` – default: `0.95`; type: `float`.
- `--repetition-penalty` – default: `1.1`; type: `float`. Values above 1.0 discourage repeating the same tokens
- `--no-repeat-ngram-size` – default: `4`; type: `int`. Block exact repeated n-grams; set to 0 to disable
- `--seed` – default: `42`; type: `int`.

## A. llm120-benchmark

Entrypoint: `llm120.benchmark:main`.

Source: `src/llm120/benchmark.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--config` – default: `Path('configs/train_125m.yaml')`; type: `Path`.
- `--steps` – default: `10`; type: `int`.
- `--sequence-length` – type: `int`; no explicit default.
- `--micro-batch-size` – type: `int`; no explicit default.
- `--vocab-size` – type: `int`; no explicit default. Use a synthetic tokenizer of this size (allows benchmarking before data setup)
- `--no-checkpointing` – action: `store_true`; default: `False`.
- `--compile` – action: `store_true`; default: `False`. Benchmark `torch.compile` with the same mode supported by the trainer

## A. llm120-doctor

Entrypoint: `llm120.doctor:main`.

Source: `src/llm120/doctor.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--config` – default: `Path('configs/train_125m.yaml')`; type: `Path`.

## A. llm120-download-posttrain

Entrypoint: `llm120.posttrain_download:main`.

Source: `src/llm120/posttrain_download.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--output` – default: `Path('data/posttrain/raw')`; type: `Path`.
- `--only` – default: `all`; choices: `['all', *(item.key for item in DATASETS)]`.

## A. llm120-prepare-posttrain

Entrypoint: `llm120.posttrain_prepare:main`.

Source: `src/llm120/posttrain_prepare.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--input` – default: `Path('data/posttrain/raw')`; type: `Path`.
- `--output` – default: `Path('data/posttrain/normalized')`; type: `Path`.
- `--seed` – default: `42`; type: `int`.
- `--validation-rows` – default: `2000`; type: `int`.

## A. llm120-sft

Entrypoint: `llm120.sft:main`.

Source: `src/llm120/sft.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--model` – default: `Path('checkpoints/llm120-english')`; type: `Path`.
- `--data` – default: `Path('data/posttrain/normalized')`; type: `Path`.
- `--output` – default: `Path('checkpoints/llm120-sft')`; type: `Path`.
- `--epochs` – default: `2.0`; type: `float`.
- `--max-steps` – default: `-1`; type: `int`.
- `--resume` – default: `auto`. `auto`, `none`, or an explicit Trainer checkpoint

## A. llm120-dpo

Entrypoint: `llm120.dpo:main`.

Source: `src/llm120/dpo.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--model` – default: `Path('checkpoints/llm120-sft')`; type: `Path`.
- `--data` – default: `Path('data/posttrain/normalized')`; type: `Path`.
- `--output` – default: `Path('checkpoints/llm120-dpo')`; type: `Path`.
- `--epochs` – default: `2.0`; type: `float`.

- `--max-steps` – default: -1; type: int.
- `--resume` – default: auto.

## A. llm120-reward

Entrypoint: `llm120.reward:main`.

Source: `src/llm120/reward.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--model` – default: `Path('checkpoints/llm120-sft')`; type: `Path`.
- `--data` – default: `Path('data/posttrain/normalized')`; type: `Path`.
- `--output` – default: `Path('checkpoints/llm120-reward')`; type: `Path`.
- `--epochs` – default: 2.0; type: `float`.
- `--max-steps` – default: -1; type: `int`.
- `--resume` – default: auto.

## A. llm120-rlhf

Entrypoint: `llm120.rlhf:main`.

Source: `src/llm120/rlhf.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `--model` – default: `Path('checkpoints/llm120-dpo')`; type: `Path`.
- `--reward-model` – default: `Path('checkpoints/llm120-reward')`; type: `Path`.
- `--data` – default: `Path('data/posttrain/normalized')`; type: `Path`.
- `--output` – default: `Path('checkpoints/llm120-assistant')`; type: `Path`.
- `--max-steps` – default: 250; type: `int`.
- `--resume` – default: auto.
- `--allow-weak-reward-model` – action: `store_true`; default: `False`. Bypass the held-out preference-accuracy safety gate (not recommended)

## A. llm120-chat

Entrypoint: `llm120.chat:main`.

Source: `src/llm120/chat.py`. Values below are extracted from argparse calls; expressions are shown literally when they depend on code constants or the environment.

- `prompt` – default: Explain why the sky is blue in simple terms.; nargs: ?.
- `--model` – default: `Path('checkpoints/llm120-assistant')`; type: `Path`.
- `--system` – default: You are a helpful, honest, and concise AI assistant..
- `--max-new-tokens` – default: 256; type: `int`.
- `--temperature` – default: 0.7; type: `float`.
- `--top-p` – default: 0.9; type: `float`.
- `--greedy` – action: `store_true`; default: `False`.
- `--seed` – default: 42; type: `int`.

## A. Standalone tokenizer comparison

Source: `scripts/compare_tokenizers.py`. Values below are extracted from `argparse` calls; expressions are shown literally when they depend on code constants or the environment.

- `--tokenizer-a` – type: Path; required: True; no explicit default.
- `--tokenizer-b` – type: Path; required: True; no explicit default.
- `--data` – type: Path; required: True; no explicit default.
- `--output` – type: Path; required: True; no explicit default.
- `--samples-per-domain` – default: 50000; type: int.
- `--tail-start-row` – default: 90000; type: int.
- `--max-chars` – default: 16384; type: int.
- `--batch-size` – default: 128; type: int.
- `--seed` – default: 120; type: int.

## B. Every shipped YAML configuration

These are the complete source configurations, including comments. They are reproduced to make all defaults reviewable; the explanatory chapters take precedence where a comment is incomplete. Relative paths are resolved against the config file’s grandparent.

### B. pilot.yaml

```
project:
  name: llm120-pilot
  output_dir: checkpoints/llm120-pilot
  seed: 42
paths:
  tokenizer: artifacts/tokenizer-pilot
  train_manifest: data/tokenized-pilot/train-manifest.json
  validation_manifest: data/tokenized-pilot/validation-manifest.json
model:
  hidden_size: 576
  intermediate_size: 1536
  num_hidden_layers: 30
  num_attention_heads: 9
  num_key_value_heads: 3
  max_position_embeddings: 2048
  rope_theta: 10000.0
  rms_norm_eps: 1.0e-5
  initializer_range: 0.041666666666666664
  tie_word_embeddings: true
training:
  sequence_length: 2048
  micro_batch_size: 4
  gradient_accumulation_steps: 8
  max_tokens: 1310720
  learning_rate: 0.003
  warmup_steps: 10
  decay_ratio: 0.20
  min_learning_rate_ratio: 0.0
  adam_beta1: 0.9
  adam_beta2: 0.95
  adam_epsilon: 1.0e-8
  weight_decay: 0.01
  max_grad_norm: 1.0
  precision: bf16
  gradient_checkpointing: true
  torch_compile: false
  allow_tf32: true
  fused_optimizer: true
  log_every_steps: 1
```

```
eval_every_steps: 10
eval_batches: 16
save_every_steps: 10
keep_last_checkpoints: 2
```

## B. train\_125m.yaml

```
project:
  name: llm120-english
  output_dir: checkpoints/llm120-english
  seed: 42

paths:
  tokenizer: artifacts/tokenizer-32k
  train_manifest: data/tokenized/train-manifest.json
  validation_manifest: data/tokenized/validation-manifest.json

model:
  # 125,077,824 parameters with a 32,768-token tied embedding.
  hidden_size: 576
  intermediate_size: 1536
  num_hidden_layers: 30
  num_attention_heads: 9
  num_key_value_heads: 3
  max_position_embeddings: 2048
  rope_theta: 10000.0
  rms_norm_eps: 1.0e-5
  initializer_range: 0.041666666666666664
  tie_word_embeddings: true

training:
  sequence_length: 2048
  micro_batch_size: 4
  gradient_accumulation_steps: 8
  # 10B tokens / (2048 * 4 * 8) = 152,588 optimizer steps.
  max_tokens: 10000000000
  learning_rate: 0.003
  warmup_steps: 2000
  decay_ratio: 0.20
  min_learning_rate_ratio: 0.0
  adam_beta1: 0.9
  adam_beta2: 0.95
  adam_epsilon: 1.0e-8
  weight_decay: 0.01
  max_grad_norm: 1.0
  precision: bf16
  gradient_checkpointing: true
  torch_compile: false
  allow_tf32: true
  fused_optimizer: true
  log_every_steps: 10
  eval_every_steps: 500
  eval_batches: 32
  save_every_steps: 1000
  keep_last_checkpoints: 3
```

## B. train\_300m.yaml

```
project:
  name: llm120-english-300m
  output_dir: checkpoints/llm120-english-300m
  seed: 42

paths:
  tokenizer: artifacts/tokenizer-32k
  train_manifest: data/tokenized/train-manifest.json
  validation_manifest: data/tokenized/validation-manifest.json

model:
  # 299,983,200 parameters with a 32,768-token tied embedding.
```

```

hidden_size: 800
intermediate_size: 2816
num_hidden_layers: 33
num_attention_heads: 10
num_key_value_heads: 2
max_position_embeddings: 2048
rope_theta: 10000.0
rms_norm_eps: 1.0e-5
initializer_range: 0.035355339059327376
tie_word_embeddings: true

training:
  sequence_length: 2048
  micro_batch_size: 1
  gradient_accumulation_steps: 32
  # 10B tokens / (2048 * 1 * 32) = 152,588 optimizer steps.
  # This is 33.3 tokens per parameter for the 299,983,200-parameter model.
  max_tokens: 10000000000
  learning_rate: 0.0015
  warmup_steps: 2000
  decay_ratio: 0.20
  min_learning_rate_ratio: 0.0
  adam_beta1: 0.9
  adam_beta2: 0.95
  adam_epsilon: 1.0e-8
  weight_decay: 0.01
  max_grad_norm: 1.0
  precision: bf16
  gradient_checkpointing: true
  torch_compile: false
  allow_tf32: true
  fused_optimizer: true
  log_every_steps: 10
  eval_every_steps: 500
  eval_batches: 32
  save_every_steps: 1000
  keep_last_checkpoints: 3

```

## B. train\_350m.yaml

```

project:
  name: llm120-english-350m
  output_dir: checkpoints/llm120-english-350m
  seed: 42

paths:
  tokenizer: artifacts/tokenizer-32k
  train_manifest: data/tokenized/train-manifest.json
  validation_manifest: data/tokenized/validation-manifest.json

model:
  # 349,928,800 parameters with a 32,768-token tied embedding.
  hidden_size: 800
  intermediate_size: 2112
  num_hidden_layers: 49
  num_attention_heads: 10
  num_key_value_heads: 2
  max_position_embeddings: 2048
  rope_theta: 10000.0
  rms_norm_eps: 1.0e-5
  initializer_range: 0.035355339059327376
  tie_word_embeddings: true

training:
  sequence_length: 2048
  micro_batch_size: 1
  gradient_accumulation_steps: 32
  # 10B tokens / (2048 * 1 * 32) = 152,588 optimizer steps.
  # This is 28.6 tokens per parameter for the 349,928,800-parameter model.
  max_tokens: 10000000000
  learning_rate: 0.0015
  warmup_steps: 2000
  decay_ratio: 0.20

```

```
min_learning_rate_ratio: 0.0
adam_beta1: 0.9
adam_beta2: 0.95
adam_epsilon: 1.0e-8
weight_decay: 0.01
max_grad_norm: 1.0
precision: bf16
gradient_checkpointing: true
torch_compile: false
allow_tf32: true
fused_optimizer: true
log_every_steps: 10
eval_every_steps: 500
eval_batches: 32
save_every_steps: 1000
keep_last_checkpoints: 3
```

## B. train\_370m.yaml

```
project:
  name: llm120-english-370m
  output_dir: checkpoints/llm120-english-370m
  seed: 42

paths:
  tokenizer: artifacts/tokenizer-32k
  train_manifest: data/tokenized/train-manifest.json
  validation_manifest: data/tokenized/validation-manifest.json

model:
  # 369,987,904 parameters with a 32,768-token tied embedding.
  hidden_size: 832
  intermediate_size: 2496
  num_hidden_layers: 44
  num_attention_heads: 8
  num_key_value_heads: 1
  max_position_embeddings: 2048
  rope_theta: 10000.0
  rms_norm_eps: 1.0e-5
  initializer_range: 0.03466876226407682
  tie_word_embeddings: true

training:
  sequence_length: 2048
  micro_batch_size: 1
  gradient_accumulation_steps: 32
  # 10B tokens / (2048 * 1 * 32) = 152,588 optimizer steps.
  # This is 27.0 tokens per parameter for the 369,987,904-parameter model.
  max_tokens: 10000000000
  learning_rate: 0.0015
  warmup_steps: 2000
  decay_ratio: 0.20
  min_learning_rate_ratio: 0.0
  adam_beta1: 0.9
  adam_beta2: 0.95
  adam_epsilon: 1.0e-8
  weight_decay: 0.01
  max_grad_norm: 1.0
  precision: bf16
  gradient_checkpointing: true
  torch_compile: false
  allow_tf32: true
  fused_optimizer: true
  log_every_steps: 10
  eval_every_steps: 500
  eval_batches: 32
  save_every_steps: 1000
  keep_last_checkpoints: 3
```

## B. train\_400m.yaml

```

project:
  name: llm120-english-400m
  output_dir: checkpoints/llm120-english-400m
  seed: 42

paths:
  tokenizer: artifacts/tokenizer-32k
  train_manifest: data/tokenized/train-manifest.json
  validation_manifest: data/tokenized/validation-manifest.json

model:
  # 404,827,200 parameters with a 32,768-token tied embedding.
  hidden_size: 960
  intermediate_size: 3328
  num_hidden_layers: 31
  num_attention_heads: 12
  num_key_value_heads: 4
  max_position_embeddings: 2048
  rope_theta: 10000.0
  rms_norm_eps: 1.0e-5
  initializer_range: 0.03227486121839514
  tie_word_embeddings: true

training:
  sequence_length: 2048
  micro_batch_size: 1
  gradient_accumulation_steps: 32
  # 10B tokens / (2048 * 1 * 32) = 152,588 optimizer steps.
  # This is 24.7 tokens per parameter for the 404,827,200-parameter model.
  max_tokens: 10000000000
  learning_rate: 0.0015
  warmup_steps: 2000
  decay_ratio: 0.20
  min_learning_rate_ratio: 0.0
  adam_beta1: 0.9
  adam_beta2: 0.95
  adam_epsilon: 1.0e-8
  weight_decay: 0.01
  max_grad_norm: 1.0
  precision: bf16
  gradient_checkpointing: true
  torch_compile: false
  allow_tf32: true
  fused_optimizer: true
  log_every_steps: 10
  eval_every_steps: 500
  eval_batches: 32
  save_every_steps: 1000
  keep_last_checkpoints: 3

```

## C. Post-training constructor settings

The base YAML does not control these settings. This appendix extracts every keyword passed to SFTConfig, DPOConfig, RewardConfig, and RLOOConfig. Shared warmup calculations are shown as expressions; see Chapter 12 for their meaning.

### C. sft.py

```

output_dir = str(args.output)
num_train_epochs = args.epochs
max_steps = args.max_steps
per_device_train_batch_size = 2
per_device_eval_batch_size = 2
gradient_accumulation_steps = 8
learning_rate = 0.0003
lr_scheduler_type = 'cosine'
warmup_steps = linear_warmup_steps(rows=len(train), epochs=args.epochs, batch_size=2, accumulation_steps=8,
    ↳ max_steps=args.max_steps)
weight_decay = 0.01

```

```

adam_beta1 = 0.9
adam_beta2 = 0.95
max_grad_norm = 1.0
bf16 = True
tf32 = True
gradient_checkpointing = False
gradient_checkpointing_kwargs = {'use_reentrant': False}
optim = 'adamw_torch_fused'
max_length = 2048
assistant_only_loss = True
packing = False
eval_strategy = 'steps'
eval_steps = 500
logging_steps = 10
logging_first_step = True
save_steps = 500
save_total_limit = 3
report_to = 'tensorboard'
dataset_num_proc = 12
dataloader_num_workers = 4
seed = 42
data_seed = 42

```

## C. dp0.py

```

output_dir = str(args.output)
num_train_epochs = args.epochs
max_steps = args.max_steps
per_device_train_batch_size = 1
per_device_eval_batch_size = 1
gradient_accumulation_steps = 16
learning_rate = 1e-06
lr_scheduler_type = 'cosine'
warmup_steps = linear_warmup_steps(rows=len(train), epochs=args.epochs, batch_size=1, accumulation_steps=16,
    ↳ max_steps=args.max_steps)
weight_decay = 0.0
adam_beta1 = 0.9
adam_beta2 = 0.95
max_grad_norm = 1.0
bf16 = True
tf32 = True
gradient_checkpointing = True
gradient_checkpointing_kwargs = {'use_reentrant': False}
optim = 'adamw_torch_fused'
max_length = 1024
beta = 0.5
loss_type = ['sigmoid']
precompute_ref_log_probs = True
precompute_ref_batch_size = 2
eval_strategy = 'steps'
eval_steps = 500
logging_steps = 10
logging_first_step = True
save_steps = 500
save_total_limit = 3
report_to = 'tensorboard'
dataset_num_proc = 12
dataloader_num_workers = 4
seed = 42
data_seed = 42

```

## C. reward.py

```

output_dir = str(args.output)
num_train_epochs = args.epochs
max_steps = args.max_steps
per_device_train_batch_size = 2
per_device_eval_batch_size = 2
gradient_accumulation_steps = 8
learning_rate = 2e-05

```

```

lr_scheduler_type = 'cosine'
warmup_steps = linear_warmup_steps(rows=len(train), epochs=args.epochs, batch_size=2, accumulation_steps=8,
    ↪ max_steps=args.max_steps)
weight_decay = 0.01
adam_beta1 = 0.9
adam_beta2 = 0.95
max_grad_norm = 1.0
bf16 = True
tf32 = True
gradient_checkpointing = True
gradient_checkpointing_kwargs = {'use_reentrant': False}
optim = 'adamw_torch_fused'
max_length = 1024
center_rewards_coefficient = 0.01
eval_strategy = 'steps'
eval_steps = 250
logging_steps = 10
logging_first_step = True
save_steps = 250
save_total_limit = 3
report_to = 'tensorboard'
dataset_num_proc = 12
dataloader_num_workers = 4
seed = 42
data_seed = 42

```

## C. rlhf.py

```

output_dir = str(args.output)
max_steps = args.max_steps
per_device_train_batch_size = 2
per_device_eval_batch_size = 2
gradient_accumulation_steps = 8
learning_rate = 1e-06
lr_scheduler_type = 'cosine'
warmup_steps = linear_warmup_steps(rows=len(train), epochs=1.0, batch_size=2, accumulation_steps=8, max_steps=
    ↪ args.max_steps)
adam_beta1 = 0.9
adam_beta2 = 0.95
max_grad_norm = 1.0
bf16 = True
tf32 = True
gradient_checkpointing = True
gradient_checkpointing_kwargs = {'use_reentrant': False}
optim = 'adamw_torch_fused'
model_init_kwargs = {'dtype': torch.float32, 'attn_implementation': 'sdpa'}
num_generations = 2
max_completion_length = 128
temperature = 0.8
top_p = 0.95
repetition_penalty = 1.05
beta = 0.1
num_iterations = 1
epsilon = 0.2
reward_weights = [1.0, 0.1]
normalize_advantages = True
mask_truncated_completions = True
eval_strategy = 'steps'
eval_steps = 50
logging_steps = 5
logging_first_step = True
log_completions = True
num_completions_to_print = 4
save_steps = 50
save_total_limit = 3
report_to = 'tensorboard'
dataloader_num_workers = 2
seed = 42
data_seed = 42

```

## D. Complete Python symbol index

Every function, class, method, and directly nested helper in the main package, comparison script, and tests is listed below. Line ranges refer to this source snapshot. The duplicate Colab files share the same definitions and line numbers. Chapter 18 explains module responsibilities; this index is for locating implementation details.

### D. `src/llm120/___init___`.py

Package metadata only; no function or class definitions.

### D. `src/llm120/benchmark`.py

- `parse_args` – function, lines 15-32.
- `main` – function, lines 35-98.

### D. `src/llm120/chat`.py

- `parse_args` – function, lines 12-24.
- `main` – function, lines 27-60.

### D. `src/llm120/checkpoint`.py

- `checkpoints` – function, lines 17-25.
- `latest_checkpoint` – function, lines 28-30.
- `capture_rng_state` – function, lines 33-41.
- `restore_rng_state` – function, lines 44-49.
- `save_checkpoint` – function, lines 52-88.
- `load_trainer_state` – function, lines 91-102.

### D. `src/llm120/config`.py

- `ProjectConfig` – class, lines 12-15.
- `PathsConfig` – class, lines 19-22.
- `ModelConfig` – class, lines 26-36.
- `TrainingConfig` – class, lines 40-71.
- `TrainingConfig.tokens_per_step` – function, lines 66-67.
- `TrainingConfig.max_steps` – function, lines 70-71.
- `Config` – class, lines 75-80.
- `_resolve` – function, lines 83-85.
- `load_config` – function, lines 88-105.
- `validate_config` – function, lines 108-122.

### D. `src/llm120/data`.py

- `TokenShard` – class, lines 15-18.
- `TokenBlockCorpus` – class, lines 21-108. Random-access fixed-length blocks backed by uint16 memory maps.
- `TokenBlockCorpus.__init__` – function, lines 29-61.

- `TokenBlockCorpus._permutation` – function, lines 63-68.
- `TokenBlockCorpus.permuted_index` – function, lines 70-75.
- `TokenBlockCorpus._array` – function, lines 77-87.
- `TokenBlockCorpus.block` – function, lines 89-101.
- `TokenBlockCorpus.batch` – function, lines 103-104.
- `TokenBlockCorpus.usable_tokens` – function, lines 107-108.
- `load_manifest` – function, lines 111-112.

#### D. `src/llm120/doctor.py`

- `validate_manifest` – function, lines 16-44.
- `parse_args` – function, lines 47-50.
- `main` – function, lines 53-92.

#### D. `src/llm120/download.py`

- `parse_args` – function, lines 14-25.
- `evenly_spaced` – function, lines 28-36.
- `main` – function, lines 39-83.

#### D. `src/llm120/dpo.py`

- `parse_args` – function, lines 22-30.
- `main` – function, lines 33-97.

#### D. `src/llm120/generate copy.py`

- `parse_args` – function, lines 12-22.
- `main` – function, lines 25-57.

#### D. `src/llm120/generate.py`

- `parse_args` – function, lines 12-40.
- `main` – function, lines 43-80.

#### D. `src/llm120/model.py`

- `make_hf_config` – function, lines 8-30.
- `make_model` – function, lines 33-34.
- `parameter_counts` – function, lines 37-40.
- `optimizer_groups` – function, lines 43-57. Decay matrix weights, excluding embeddings and all norm/scalar parameters.

## D. src/llm120/posttrain\_common.py

- `require_cuda` – function, lines 14-21.
- `resolve_model` – function, lines 24-37.
- `load_tokenizer` – function, lines 40-46.
- `load_split` – function, lines 49-53.
- `evaluation_subset` – function, lines 56-57.
- `linear_warmup_steps` – function, lines 60-69. Resolve a 10% warmup without relying on Transformers' deprecated `warmup_ratio`.
- `resolve_resume` – function, lines 72-83.
- `finish_stage` – function, lines 86-95.

## D. src/llm120/posttrain\_download.py

- `DatasetSpec` – class, lines 13-17.
- `parse_args` – function, lines 42-48.
- `repo_size` – function, lines 51-52.
- `main` – function, lines 55-94.

## D. src/llm120/posttrain\_prepare.py

- `parse_args` – function, lines 15-21.
- `parquet_files` – function, lines 24-28.
- `load_sft` – function, lines 31-49.
- `valid_messages` – function, lines 52-62.
- `explicit_preference` – function, lines 65-78.
- `load_dpo` – function, lines 81-98.
- `helpsteer_file` – function, lines 101-120.
- `normalize_helpsteer` – function, lines 123-139.
- `load_helpsteer` – function, lines 142-181.
- `load_helpsteer.unique_prompts` – function, lines 168-176.
- `write_parquet` – function, lines 184-190.
- `main` – function, lines 193-219.

## D. src/llm120/prepare.py

- `PrepareTask` – class, lines 23-30.
- `parse_args` – function, lines 33-42.
- `_init_worker` – function, lines 45-57.
- `_is_validation` – function, lines 60-64.
- `_flush` – function, lines 67-74.
- `_process` – function, lines 77-151.
- `_tokenizer_hash` – function, lines 154-155.
- `_write_manifest` – function, lines 158-177.
- `main` – function, lines 180-217.

## D. src/llm120/reward.py

- `parse_args` – function, lines 21-29.
- `main` – function, lines 32-98.

## D. src/llm120/rlhf.py

- `parse_args` – function, lines 24-37.
- `reward_accuracy` – function, lines 40-45.
- `completion_health_reward` – function, lines 48-64. Small anti-degeneration signal; the learned human reward remains dominant.
- `main` – function, lines 67-149.

## D. src/llm120/schedule.py

- `wsd_factor` – function, lines 4-20. Warmup-stable-decay factor for the next optimizer update (zero-indexed).

## D. src/llm120/sft.py

- `parse_args` – function, lines 22-30.
- `main` – function, lines 33-95.

## D. src/llm120/tokenizer\_\_train.py

- `parse_args` – function, lines 36-44.
- `parquet_files` – function, lines 47-51.
- `text_iterator` – function, lines 54-84.
- `build_tokenizer` – function, lines 87-92.
- `main` – function, lines 95-154.

## D. src/llm120/train.py

- `parse_args` – function, lines 25-34.
- `seed_everything` – function, lines 37-41.
- `file_sha256` – function, lines 44-49.
- `precision_context` – function, lines 52-56.
- `set_learning_rate` – function, lines 59-61.
- `evaluate` – function, lines 65-83.
- `resolve_resume` – function, lines 86-94.
- `make_optimizer` – function, lines 97-110.
- `run` – function, lines 113-329.
- `run.request_stop` – function, lines 200-203.
- `main` – function, lines 332-334.

## D. scripts/compare\_\_tokenizers.py

- `parse_args` – function, lines 23-34.
- `_domain` – function, lines 37-42.
- `sample_tail_documents` – function, lines 45-100. Sample equal per-shard quotas from rows well beyond tokenizer-training prefixes.
- `DomainMetrics` – class, lines 104-110.
- `percentile` – function, lines 113-120.
- `evaluate` – function, lines 123-163.
- `summarize_domain` – function, lines 166-201.
- `comparison` – function, lines 204-227.
- `markdown_report` – function, lines 230-282.
- `main` – function, lines 285-343.

## D. tests/test\_\_data.py

- `make_manifest` – function, lines 9-22.
- `test_blocks_are_a_full_permutation_each_epoch` – function, lines 25-34.
- `test_block_contents_and_resume_are_stateless` – function, lines 37-41.

## D. tests/test\_\_doctor.py

- `test_validate_manifest_checks_tokenizer_and_binary_size` – function, lines 7-26.

## D. tests/test\_\_download.py

- `test_evenly_spaced_includes_endpoints` – function, lines 4-7.

## D. tests/test\_\_posttrain.py

- `test_explicit_preference_removes_shared_prompt` – function, lines 6-15.
- `test_helpsteer_preference_sign` – function, lines 18-29.
- `test_completion_health_penalizes_repetition` – function, lines 32-35.
- `test_helpsteer_file_ignores_hub_lock` – function, lines 38-45.
- `test_linear_warmup_uses_override_or_dataset_size` – function, lines 48-57.

## D. tests/test\_\_schedule.py

- `test_wsd_schedule` – function, lines 6-15.

## E. Colab cell-by-cell reference

Indices below are zero-based notebook cell indices. This is the complete cell inventory, including prose cells. Notebook code syntax is checked without executing Drive or GPU operations.

- Cell 0 (markdown): Overview and historical runtime/storage assumptions.
- Cell 1 (code): User settings; model selection; token budget; batch arithmetic; Drive paths.

- Cell 2 (markdown): Mount section heading.
- Cell 3 (code): Mount Drive, verify project folder, create directories, inspect filesystem free space.
- Cell 4 (markdown): Dependency-installation explanation.
- Cell 5 (code): Install requirements, import torch/Transformers, select BF16 or FP16 from hardware.
- Cell 6 (code): Set package import path and PYTHONPATH; define shell helper and optional HF\_TOKEN forwarding.
- Cell 7 (markdown): Data-caching explanation.
- Cell 8 (code): Reuse Drive corpus or download estimated shards, train tokenizer, prepare locally, copy to Drive.
- Cell 9 (markdown): Config-identity explanation.
- Cell 10 (code): Generate model/training YAML with sorted keys; run doctor (T4 blocker described in Chapter 13).
- Cell 11 (markdown): Benchmark explanation; its prose scaling instruction is imprecise – use token budget divided by throughput.
- Cell 12 (code): Run synthetic benchmark for 20 measured iterations after warmup.
- Cell 13 (markdown): Training/resume explanation.
- Cell 14 (code): Run trainer with automatic resume.
- Cell 15 (markdown): Generation explanation.
- Cell 16 (code): Generate a base-model continuation from the Drive output directory.
- Cell 17 (markdown): Later-session checklist, browser reconnect snippet, and incomplete retained-only storage estimate.

## E. Source-copy equivalence

All 21 of 21 shared Python files are byte-identical. The main-only `generatecopy.py` is not part of the Colab copy. A change to one source tree will not automatically update the other.

## F. Packaging and recorded dependencies

### F. pyproject.toml

```
[project]
name = "llm120"
version = "0.1.0"
description = "A reproducible single-GPU pipeline for pretraining a 125M parameter language model from scratch"
readme = "README.md"
requires-python = ">=3.11,<3.13"
dependencies = [
    "huggingface-hub>=0.34,<2",
    "numpy>=2.0,<3",
    "pyarrow>=20,<24",
    "pyyaml>=6.0,<7",
    "safetensors>=0.5,<1",
    "tensorboard>=2.19,<3",
    "tokenizers>=0.21,<1",
    "torch>=2.7,<3",
    "transformers>=4.53,<6",
]

[project.optional-dependencies]
dev = ["pytest>=8.3,<10", "ruff>=0.12,<1"]
posttrain = [
    "accelerate>=1.10,<2",
    "datasets>=4.0,<5",
    "trl>=1.6,<2",
]
```

```

]

[project.scripts]
llm120-download = "llm120.download:main"
llm120-tokenizer = "llm120.tokenizer_train:main"
llm120-prepare = "llm120.prepare:main"
llm120-train = "llm120.train:main"
llm120-generate = "llm120.generate:main"
llm120-benchmark = "llm120.benchmark:main"
llm120-doctor = "llm120.doctor:main"
llm120-download-posttrain = "llm120.posttrain_download:main"
llm120-prepare-posttrain = "llm120.posttrain_prepare:main"
llm120-sft = "llm120.sft:main"
llm120-dpo = "llm120.dpo:main"
llm120-reward = "llm120.reward:main"
llm120-rlhf = "llm120.rlhf:main"
llm120-chat = "llm120.chat:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/llm120"]

[[tool.uv.index]]
name = "pytorch-cu128"
url = "https://download.pytorch.org/whl/cu128"
explicit = true

[tool.uv.sources]
torch = { index = "pytorch-cu128" }

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.ruff]
line-length = 100
target-version = "py311"
extend-exclude = ["data", "artifacts", "checkpoints"]

```

## F. Colab requirements

```

# Runtime dependencies for LLM120 on Google Colab.
# PyTorch is NOT listed here: Colab ships its own CUDA-enabled torch, and we
# deliberately avoid the repo's cu128 index (that index is for the author's
# Blackwell RTX 5050, sm_120 -- wrong for Colab's T4, sm_75).
transformers>=4.53,<6
tokenizers>=0.21
huggingface-hub>=0.34
pyarrow>=20
safetensors>=0.5
tensorboard>=2.19
pyyaml>=6.0

```

## F. Lockfile package inventory

Lock format version 1, revision 3; Python constraint  $\geq 3.11, < 3.13$ . There are 91 package entries. Duplicate package names reflect alternate resolutions. Artifact URLs, package hashes, dependency edges, and environment markers remain in uv.lock; this table is its complete name/version inventory, not an installed-package report.

| Package          | Version | Package              | Version   |
|------------------|---------|----------------------|-----------|
| absl-py          | 2.5.0   | nvidia-cufile-cu12   | 1.13.1.3  |
| accelerate       | 1.14.0  | nvidia-curand-cu12   | 10.3.9.90 |
| aiohappyeyeballs | 2.7.1   | nvidia-cusolver-cu12 | 11.7.3.90 |
| aiohttp          | 3.14.2  | nvidia-cuspars-cu12  | 12.5.8.93 |

| Package                  | Version   | Package                 | Version      |
|--------------------------|-----------|-------------------------|--------------|
| aiohttp                  | 1.4.0     | nvidia-cusparselt-cu12  | 0.7.1        |
| annotated-doc            | 0.0.4     | nvidia-nccl-cu12        | 2.28.9       |
| anyio                    | 4.14.2    | nvidia-nvjitlink-cu12   | 12.8.93      |
| attrs                    | 26.1.0    | nvidia-nvshmem-cu12     | 3.4.5        |
| certifi                  | 2026.6.17 | nvidia-nvtx-cu12        | 12.8.90      |
| charset-normalizer       | 3.4.9     | packaging               | 26.2         |
| click                    | 8.4.2     | pandas                  | 3.0.3        |
| colorama                 | 0.4.6     | pillow                  | 12.3.0       |
| cuda-bindings            | 12.9.7    | pluggy                  | 1.6.0        |
| cuda-pathfinder          | 1.5.6     | propcache               | 0.5.2        |
| cuda-toolkit             | 12.8.1    | protobuf                | 7.35.1       |
| datasets                 | 4.8.5     | psutil                  | 7.2.2        |
| dill                     | 0.4.1     | pyarrow                 | 23.0.1       |
| filelock                 | 3.31.2    | pygments                | 2.20.0       |
| frozenset                | 1.8.0     | pytest                  | 9.1.1        |
| fsspec                   | 2026.2.0  | python-dateutil         | 2.9.0.post0  |
| grpcio                   | 1.82.1    | pyyaml                  | 6.0.3        |
| h11                      | 0.16.0    | regex                   | 2026.7.19    |
| hf-xet                   | 1.5.2     | requests                | 2.34.2       |
| httpcore                 | 1.0.9     | rich                    | 15.0.0       |
| httpx                    | 0.28.1    | ruff                    | 0.15.22      |
| huggingface-hub          | 1.24.0    | safetensors             | 0.8.0        |
| idna                     | 3.18      | setuptools              | 81.0.0       |
| iniconfig                | 2.3.0     | shellingham             | 1.5.4        |
| jinja2                   | 3.1.6     | six                     | 1.17.0       |
| llm120                   | 0.1.0     | sympy                   | 1.14.0       |
| markdown                 | 3.10.2    | tensorboard             | 2.21.0       |
| markdown-it-py           | 4.2.0     | tensorboard-data-server | 0.7.2        |
| markupsafe               | 3.0.3     | tokenizers              | 0.22.2       |
| mdurl                    | 0.1.2     | torch                   | 2.11.0+cu128 |
| mpmath                   | 1.3.0     | tqdm                    | 4.69.0       |
| multidict                | 6.7.1     | transformers            | 5.14.1       |
| multiprocess             | 0.70.19   | triton                  | 3.6.0        |
| networkx                 | 3.6.1     | trl                     | 1.8.0        |
| numpy                    | 2.4.6     | typer                   | 0.27.0       |
| numpy                    | 2.5.1     | typing-extensions       | 4.16.0       |
| nvidia-cublas-cu12       | 12.8.4.1  | tzdata                  | 2026.3       |
| nvidia-cuda-cupti-cu12   | 12.8.90   | urllib3                 | 2.7.0        |
| nvidia-cuda-nvrtc-cu12   | 12.8.93   | werkzeug                | 3.1.8        |
| nvidia-cuda-runtime-cu12 | 12.8.90   | xxhash                  | 3.8.1        |
| nvidia-cudnn-cu12        | 9.19.0.56 | yaml                    | 1.24.5       |
| nvidia-cufft-cu12        | 11.3.3.83 |                         |              |

## G. Review coverage and source fingerprints

Reviewed 61 original files. Syntax parsed 49 Python files and 9 notebook code cells without importing their dependencies. The standalone schedule test passed; full pytest collection reported four import errors. These results do not establish ML runtime compatibility or model quality.

SHA-256 prefixes below identify this snapshot. Full hashes and byte sizes are in `docs/source_inventory.json`. Generated documentation files are excluded from this original-source inventory.

| Original file                          | Bytes | SHA-256 prefix   |
|----------------------------------------|-------|------------------|
| README.md                              | 14945 | 444daae0d0fbb99d |
| colab/LLM120_Colab.ipynb               | 15998 | 8cf8da2799023a05 |
| colab/README.md                        | 6716  | 3fecbba5a091c793 |
| colab/requirements-colab.txt           | 387   | 9d441d50718d1e23 |
| colab/src/llm120/__init__.py           | 75    | 8b007b411785e954 |
| colab/src/llm120/benchmark.py          | 4035  | bc06f928a7811142 |
| colab/src/llm120/chat.py               | 2432  | ee8614e9809e3405 |
| colab/src/llm120/checkpoint.py         | 2943  | 292183dd5c35c287 |
| colab/src/llm120/config.py             | 3482  | 6558eae95c02813c |
| colab/src/llm120/data.py               | 4220  | 4d755f4d2c8f8203 |
| colab/src/llm120/doctor.py             | 4145  | 87d26ca0c5678a50 |
| colab/src/llm120/download.py           | 3115  | f66bec860f7b2a2f |
| colab/src/llm120/dpo.py                | 3154  | 0f1333b5ae012ddc |
| colab/src/llm120/generate.py           | 3299  | 20276127d16731ee |
| colab/src/llm120/model.py              | 2211  | 6e486059c9caf472 |
| colab/src/llm120/posttrain_common.py   | 3592  | 11f55d1a3fa2a921 |
| colab/src/llm120/posttrain_download.py | 2970  | 544d4688413d7506 |
| colab/src/llm120/posttrain_prepare.py  | 8848  | 9de7df8afa551cab |
| colab/src/llm120/prepare.py            | 8758  | d890bc5cad84084d |
| colab/src/llm120/reward.py             | 3293  | 685bc04b69a7679d |
| colab/src/llm120/rlhf.py               | 5658  | 24e591ffd69d880a |
| colab/src/llm120/schedule.py           | 667   | b7af493c395ef4fc |
| colab/src/llm120/sft.py                | 3154  | 55fda42b5296cc4a |
| colab/src/llm120/tokenizer_train.py    | 6099  | 51522cb6425678a7 |
| colab/src/llm120/train.py              | 13118 | 9454bdc01532ea8e |
| configs/pilot.yaml                     | 1061  | 5d64dc9ec40ffc33 |
| configs/train_125m.yaml                | 1186  | c4006f55be071f97 |
| configs/train_300m.yaml                | 1275  | 020c1f07d3784906 |
| configs/train_350m.yaml                | 1275  | 052bfe2b82be7b24 |
| configs/train_370m.yaml                | 1273  | 42d84a089f3bc95f |
| configs/train_400m.yaml                | 1274  | 0697885f293402ab |
| pyproject.toml                         | 1640  | 90cfc9f59ad80fba |
| scripts/compare_tokenizers.py          | 14434 | bcf78ea7067354d5 |
| src/llm120/__init__.py                 | 75    | 8b007b411785e954 |
| src/llm120/benchmark.py                | 4035  | bc06f928a7811142 |
| src/llm120/chat.py                     | 2432  | ee8614e9809e3405 |
| src/llm120/checkpoint.py               | 2943  | 292183dd5c35c287 |
| src/llm120/config.py                   | 3482  | 6558eae95c02813c |
| src/llm120/data.py                     | 4220  | 4d755f4d2c8f8203 |
| src/llm120/doctor.py                   | 4145  | 87d26ca0c5678a50 |
| src/llm120/download.py                 | 3115  | f66bec860f7b2a2f |
| src/llm120/dpo.py                      | 3154  | 0f1333b5ae012ddc |
| src/llm120/generatecopy.py             | 2523  | c245ef973514b060 |
| src/llm120/generate.py                 | 3299  | 20276127d16731ee |
| src/llm120/model.py                    | 2211  | 6e486059c9caf472 |
| src/llm120/posttrain_common.py         | 3592  | 11f55d1a3fa2a921 |
| src/llm120/posttrain_download.py       | 2970  | 544d4688413d7506 |
| src/llm120/posttrain_prepare.py        | 8848  | 9de7df8afa551cab |
| src/llm120/prepare.py                  | 8758  | d890bc5cad84084d |
| src/llm120/reward.py                   | 3293  | 685bc04b69a7679d |
| src/llm120/rlhf.py                     | 5658  | 24e591ffd69d880a |
| src/llm120/schedule.py                 | 667   | b7af493c395ef4fc |
| src/llm120/sft.py                      | 3154  | 55fda42b5296cc4a |

| Original file                 | Bytes  | SHA-256 prefix   |
|-------------------------------|--------|------------------|
| src/llm120/tokenizer_train.py | 6099   | 51522cb6425678a7 |
| src/llm120/train.py           | 13118  | 9454bdc01532ea8e |
| tests/test_data.py            | 1463   | 8a0d4d7b7e854863 |
| tests/test_doctor.py          | 870    | 01c745e957f4f4a9 |
| tests/test_download.py        | 245    | 591fcec5c4df7fca |
| tests/test_posttrain.py       | 2284   | 7c45812808f99da3 |
| tests/test_schedule.py        | 469    | 08ccc00a53d0525e |
| uv.lock                       | 234925 | bac9c4cf45caf90f |

## H. Glossary and handoff checklist

| Abbreviation       | Expansion / use                                                           |
|--------------------|---------------------------------------------------------------------------|
| BPE                | Byte-pair encoding; learned text-to-token segmentation.                   |
| BOS / EOS          | Beginning/end of sequence markers.                                        |
| GQA / MQA          | Grouped-query / multi-query attention.                                    |
| RoPE               | Rotary positional embeddings.                                             |
| RMSNorm            | Root-mean-square normalization.                                           |
| SDPA               | Scaled dot-product attention implementation selected in PyTorch.          |
| BF16 / FP16 / FP32 | Floating-point formats used for compute or state.                         |
| TF32               | TensorFloat-32 acceleration setting for supported hardware.               |
| MLP                | Feed-forward multilayer perceptron within each Transformer layer.         |
| WSD                | Warmup, stable learning rate, then decay.                                 |
| SFT / DPO          | Supervised fine-tuning / direct preference optimization.                  |
| RLHF / RLOO        | Reinforcement learning from human feedback / leave-one-out online method. |
| KL                 | Divergence penalty constraining movement from a reference policy.         |
| NFC                | Unicode canonical composition normalization.                              |
| RNG                | Random-number-generator state saved for reproducible continuation.        |
| memmap             | Disk-backed array accessed without loading the full corpus into RAM.      |

A useful experiment handoff includes:

- Exact model/tokenizer files, the intended inference path, and fixed example prompts.
- Source inventory, lockfile, resolved environment, GPU/driver details, and YAML bytes.
- Raw dataset revisions, normalized-data provenance, token manifests, and measured token counts.
- Run metrics, validation method, generation settings, and comparisons before/after tuning.
- Resumable state if continuation is intended, with storage and recovery instructions.
- Known limitations, unresolved review findings, and actual checks completed.